https://hjfy.top/arxiv/2607.00272



# ASPIRE：面向机器人领域的 Agentic /Skills 发现

**Runyu Lu**[1 2 * †], **Yubo Wu**[1 3 *], **Ethan Kou**[1 4 *]
Letian Fu[1 4], Wenli Xiao[1 5], Ajay Mandlekar[1], Yinzhen Xu[1]
Guanya Shi[5], Ken Goldberg[4], Ang Chen[2], Mosharaf Chowdhury[2]
Yuke Zhu[1 †], Linxi "Jim" Fan[1 †], Guanzhi Wang[1 †]
[1] NVIDIA, [2] UMich, [3] UIUC, [4] UC Berkeley, [5] CMU
[*] Equal contribution, [†] Project leads

https://research.nvidia.com/labs/gear/aspire/

**摘要：**
传统机器人编程众所周知具有挑战性：它需要协调多模态感知、管理复杂的物理接触动力学，并处理多样化的环境配置和执行失败。
我们提出了 ASPIRE （**A**gentic **S**kill **P**rogramming through **I**terative **R**obot **E**xploration），一种用于机器人领域的持续学习系统，该系统在代码即策略范式下自主编写并优化机器人控制程序，同时将经验不断积累为可复用的技能库。ASPIRE 能够实现跨多个任务、仿真与真实世界情景以及不同机体形态的可复用技能的自动化发现。
不同于依赖固定的人工设计流水线，ASPIRE 在一个开放式的持续学习环中运行，包含三个关键组件：(1) 一个闭环机器人执行引擎，能够暴露细粒度的多模态迹（例如，感知叠加、抓取候选、运动轨迹和碰撞反馈），使智能体能够自主诊断失败、合成修复方案并验证结果；(2) 一个持续扩展的技能库，将已验证的修复方案提炼为可复用、可迁移的机器人知识；(3) 一种演化搜索过程，生成多样化的任务序列和控制程序，系统性地调试它们以探索超越单一轨迹优化的范围。随着 ASPIRE 遇到的任务越来越多，其不断增长的技能库使其适应速度显著提升。
因此，ASPIRE 在扰动条件下的操作任务上（LIBERO-Pro）相比先前方法性能提升高达 77%，在 Robosuite 的双臂交接任务上提升 72%，在长时程家庭任务上（BEHAVIOR-1K）提升高达 32%。积累的技能库还进一步实现了强大的 zero-shot 泛化能力：在代表性未见的长时程任务（LIBERO-Pro Long）上，ASPIRE 达到了 31% 的成功率，显著优于先前方法 4% 的成功率，尽管后者严重依赖测试时推理与重试。
最后，仿真中发现的技能为从仿真到现实的迁移提供了初步证据，即使在不同机体和机器人 API 的情况下，也能显著减少真实机器人编程的工作量。

## 1. 引言

软件工程智能体的最新进展表明，语言模型能够自主地检查执行迹，定位故障，修改实现，并通过与执行环境的反复交互不断改进 [Anthropic, 2025, OpenAI, 2025, anomalyco, 2025, Wang et al., 2025, Yang et al., 2024]。在机器人领域，这一范式启发了代码即策略系统，该系统将感知模块、规划 API 和控制原语组合成可执行的机器人程序 [Liang et al., 2023, Singh et al., 2023, Ahn et al., 2022, Huang et al., 2023, Mu et al., 2024, Fu et al., 2026]。由于机器人行为被显式表示为程序，原则上可以通过交互反馈进行检查、编辑、调试和优化。

然而，现有的机器人编程智能体仍受限于过于简单的执行环境，这些环境仅提供粗粒度的任务级反馈。调试机器人程序本质上具有挑战性，因为故障可能源于多个相互作用的组件，包括多模态感知、运动规划、抓取生成、接触动力学以及长时程任务协调。一次失败的执行轨迹可能表明任务未成功，但无法判断根本原因究竟是感知错误、抓取不稳、规划失误，还是后续恢复失败。缺乏细粒度的诊断迹线，智能体在确定应检查哪些证据、如何定位故障或尝试何种修复策略方面能力有限。

© 2026 NVIDIA. All rights reserved.

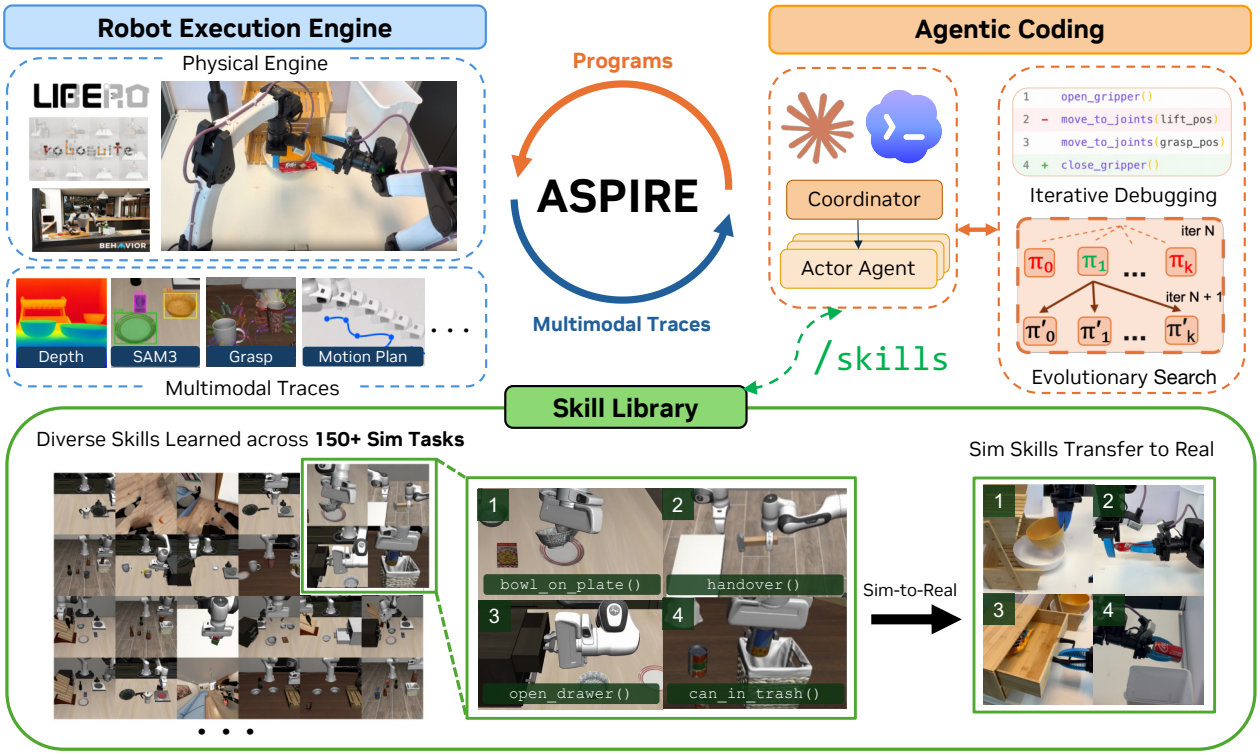



Figure 1: **Aspire 系统概览**。一个 协调器为每个任务生成一个 演员智能体（编码智能体），实现跨任务的并行学习。每个演员通过与 机器人执行引擎进行 迭代调试，不断优化和验证机器人程序，该引擎会暴露每个基本操作的多模态迹，用于失败归因与修复。演化搜索采样多样化的候选程序 $(\pi_0, \ldots, \pi_k)$，将每个程序发送至引擎，并基于存活的程序和残余失败迹来构建下一代 $(\pi'_0, \ldots, \pi'_k)$。协调器将经过验证的修复写入共享的 技能库，未来演员可将其作为上下文引导；在仿真中发现的技能也可被适配为跨具身引导，用于真实机器人的编程。

此外，现有系统无法在任务间积累经验。一旦完成一个任务，发现的修复方法和恢复策略就会被丢弃，而非整合为可复用的技能。因此，解决第 100 个任务的智能体实际上并不比解决第一个任务的智能体更有经验。

人类机器人工程师采用一种根本不同的方法。当机器人程序出现故障时，他们会回放执行过程，检查感知输出和运动轨迹，定位出错的子系统，修改实现方式，并内化可复用的恢复策略。随着时间推移，调试经验不断积累，形成可迁移的知识，包括抓取恢复启发式、导航策略、提示配方以及能泛化到多种任务的流程性修复方法。这种可复用知识的积累，正是人类机器人程序员随时间推移越来越高效的关键原因。

在本工作中，我们介绍了 Aspire ，这是一种自提升的持续学习机器人系统，能够在代码即策略范式下自主编写和优化机器人控制程序，同时将经验积累到可重用的技能库中。与在固定感知-规划-执行流水线中运行不同，Aspire 在一个开放式的学习环中运行，智能体自主决定如何检查执行迹、诊断失败、合成修复、验证修正行为，并将成功的恢复模式整合为可持久化的技能，实现跨任务迁移。

Aspire 的概述如图 1 所示。Aspire 由三个组件构建而成。首先，一个 **闭环机器人执行引擎**将粗粒度的滚动反馈替换为每个基本操作的执行轨迹。对于每一次感知、规划、抓取和控制调用，执行引擎都会记录观测结果、输入、输出以及尽可能多的视觉证据。这些丰富的多模态轨迹使智能体能够选择性地检查关键的基本操作日志，逐步定位故障，并通过重新执行来验证修复效果。其次，Aspire 维护一个不断增长的 **技能库**，将经过验证的修复方法提炼为可重用、可迁移的机器人知识，未来任务中可通过上下文引导的方式检索使用。第三，Aspire 采用一种 **演化搜索过程**，生成多样化的任务序列和控制程序，通过迭代调试与并行优化，探索超越单一轨迹自我改进的可能。这三个组件共同构建了一个自我改进的机器人系统，其性能随经验积累而提升：Aspire 见到的任务越多，其技能库越大，越能迁移到新任务、更长时序的行为以及真实世界机器人场景中，当不同实体形态下出现相似的故障-恢复模式时，系统仍能有效应对。

在实践中，Aspire 在多种短周期和长周期机器人基准测试中展现出强大的自我改进能力。相较于先前的编码智能体 [Fu et al., 2026]，Aspire 在 LIBERO-Pro [Zhou et al., 2025] 扰动套件上成功率提升高达 77 个百分点，在 Robosuite [Zhu et al., 2020] 高接触密度操作任务上提升高达 72 个百分点，在 BEHAVIOR-1K [Li et al., 2024] 具有程序化生成布局的长周期家庭任务上提升高达 32 个百分点。基于在 LIBERO-90 [Liu et al., 2023] 上积累的技能库，Aspire 实现了 zero-shot 迁移至 LIBERO-Pro Long [Zhou et al., 2025]，达到 31% 的成功率，而先前方法尽管依赖测试时推理和重试，仍饱和于 4%。我们进一步在真实双臂机器人上评估了三种仿真发现的技能，结果表明，将仿真中发现的技能作为上下文引导进行检索，可减少现实世界中的推理 token 数量，并使原本采用朴素调试而完全失败的程序成功运行。

## 2. 方法

Aspire 由三个组件构成，共同形成一个开放式的学习环：(1) 一个 *机器人执行引擎*（§2.1），用于暴露每个基本操作的多模态迹，以进行失败归因，并执行智能体编写的修复代码，实现闭环验证；(2) 一个 *技能库*（§2.2），将经过验证的修复作为可复用的技能进行累积；以及 (3) 一种 *演化搜索过程*（§2.3），使探索范围超越单一轨迹的修复。随着 Aspire 面对的任务增多，其技能库不断增长，使得未来任务能够继承已积累的修复方案和可复用的策略。

如图 1 所示，Aspire 采用协调者–演员架构。一个中心协调者管理共享的技能库，并将演员编码智能体分配至各个任务，而每个演员在机器人执行引擎内完成程序编写、执行、诊断和修复。演员之间不交换完整的聊天历史或原始回放轨迹。相反，可迁移的经验被提炼进技能库，使得每个演员的上下文窗口能够专注于当前的任务说明、当前程序以及与当前失败相关的结构化执行迹。

### 2.1. 机器人执行引擎

具身编码智能体需要执行证据来调试机器人程序。先前的方法通过固定的人工设计接口暴露这些证据，通常以手动整理的场景级摘要或一组预定义的观测值形式呈现。这带来了一个权衡：证据过少可能隐藏导致失败的基本操作，而过多的原始视觉上下文则可能使智能体偏离导致失败的因果链。

Aspire 将这一固定反馈通道转变为一个开放式的调试环境。机器人执行引擎会记录感知、规

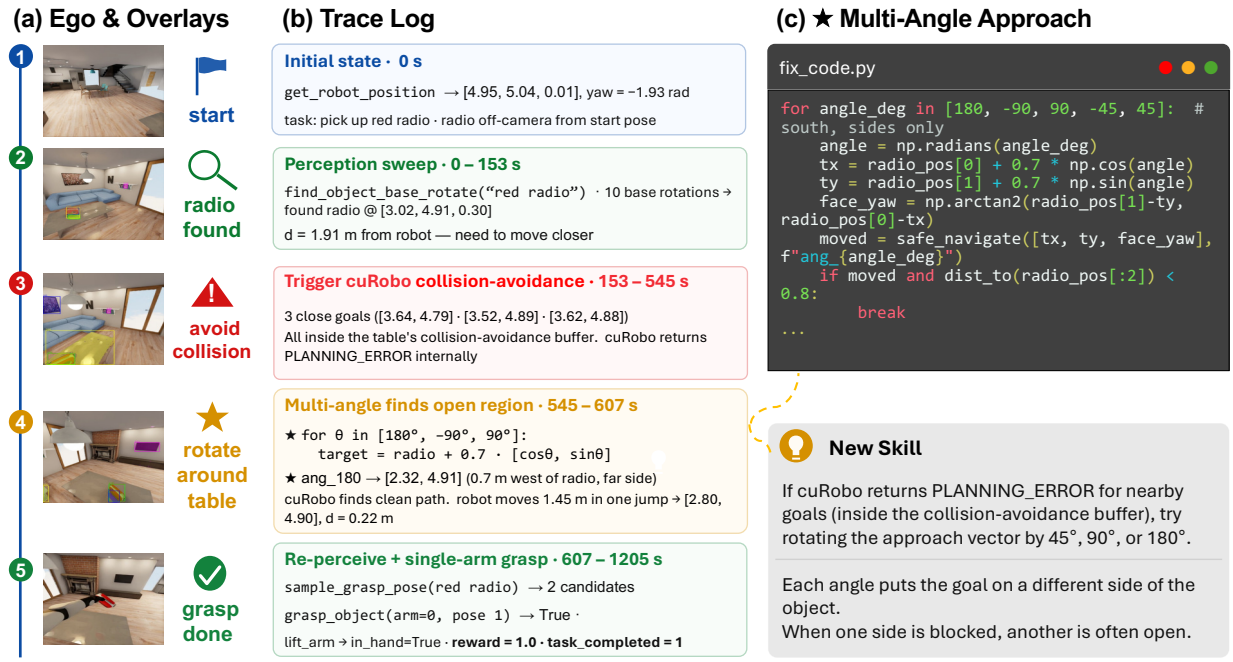



Figure 2: **机器人执行引擎**。在 BEHAVIOR-1K NAVIGATE-AND-PICK-UP-RADIO 任务上的迹引导调试。(a) 自身视角的关键帧与叠加图显示机器人定位到了收音机，但未能接近它。(b) 基本操作迹将失败定位为反复出现的 PLANNING_ERROR：候选导航目标位于桌子的避障缓冲区内。(c) 智能体通过多角度接近程序修复了问题，从可达侧重新感知收音机，并完成抓取。经验证的修复被接纳为可复用的 多角度接近技能。

划和控制调用的每个基本操作的多模态迹，将迹暴露给编码智能体，并执行智能体编写的修复代码以进行闭环验证。对于每个基本操作调用，该迹存储被调用的 API、输入与输出、返回状态以及相关的多模态证据，例如 RGB 关键帧、叠加信息、抓取候选、物体位姿和运动规划结果。智能体不会接收到完整的视频帧；引擎会保留每个基本操作调用前后立即对应的帧，以及相应的叠加信息和返回值，使智能体能够专注于与失败相关的调用周围的证据。

图 2 展示了 BEHAVIOR-1K NAVIGATE-AND-PICK-UP-RADIO 调试回合的一个局部放大。自视角关键帧显示，机器人找到了收音机，但多次尝试接近均失败，随后在改变接近方向后成功。原始迹线定位了故障原因：感知成功并返回了收音机的位姿，但重复调用 navigate_to_pose 均返回 PLANNING_ERROR。通过检查导航返回值及相关日志，智能体发现生成的导航目标过于靠近桌子边缘，距离表边约 20 厘米。这触发了碰撞规避机制，导致规划器失败。因此，故障并非由无法检测或抓取收音机引起，而是由于目标位姿在桌子的碰撞约束下不可行所致。

修复方案直接源自这一诊断。智能体并未更改感知提示或抓取原始动作，而是编写了一个多角度接近的程序，对无线电设备周围的多个替代导航目标进行采样，并执行一个能够清除碰撞缓冲区的接近方向。执行引擎暴露了证据，验证了修复后的程序，并使智能体能够分析生成的日志、形成假设并做出有针对性的修复决策。




Figure 3: **技能库**。 ASPIRE 将经过验证的、由智能体发现的修复知识以可重用的上下文内技能形式存储，而非固定的人工编写基础操作集合。顶部：代表性条目展示了关于定位消歧、运动基元构建和导航恢复的学成技能。中部：该技能库跨越多种异构类别，包括定位、导航、运动基元、物体级抓取、场景理解以及调试工作流。底部：在仿真环境中发现的部分技能被用作真实机器人编程的上下文引导，提供了技能可在不同实体间迁移的证据。更多技能示例和提示见附录 A。

### 2.2. 技能库

程序错误在不同任务间反复出现，但可复用的知识很少是整个任务的程序。ASPIRE 的技能库存储了异构的修复知识：定位启发式、感知提示、抓取约束、导航恢复策略、运动基元、场景理解过程以及调试工作流。我们不会预先规定这一分类体系。技能是从经过验证的修复中归纳得出的：编码智能体从执行迹中诊断失败，修补程序，对调试配置验证修复结果，协调器仅将可复用的模式纳入共享库中。

每种技能均以紧凑的上下文引导形式存储，包括失败特征、适用条件、修复策略，以及在有用时的代表性代码草图。图 3 展示了代表性条目及由此带来的知识库广度。对于图 2 中的无线电任务，所接纳的技能是一种导航恢复模式，而非完整的无线电拾取程序：当规划器在障碍物边界附近反复出现错误，原因是采样的目标位姿落入碰撞缓冲区时，应在重新尝试感知和抓取前，围绕物体采样替代的接近方向。这种表示方式使未来的演员能够复用已验证的修复方案，而非通过运行时推理重新发现它们，支持 zero-shot 转移至更复杂的仿真任务，并为选定的仿真中发现的技能提供跨实体泛化并迁移至真实机器人机制。

演员报告结构化结果，总结故障模式、已验证的修复方案以及可能可转移的修复模式。协调员审计这些结果，验证其是否符合允许的 API 策略，并仅将通过调试验证且可复用的修复方案纳入共享技能库。附录 E.1 提供了用于演员报告和协调员引导技能准入的提示。

### 2.3. 演化搜索

仅依靠迹引导的调试可能会陷入局部修复环，即智能体反复修补同一种失败的策略，而非探索根本不同的任务解决方法。ASPIRE 采用演化搜索来扩展可执行机器人程序的探索范围，鼓励生成多样化的修复假设和任务策略。

每轮中，基于技能库，编码智能体根据表现最佳的先前程序以及先前评估中的失败迹，提出一组 $K$ 个候选程序。每个候选程序在机器人执行引擎中执行，生成任务结果以及新的诊断迹。下一轮则基于表现最佳的程序及其剩余的故障模式进行条件化，从而允许搜索探索不同的策略，而非反复优化同一解决方案。

搜索目标是机器人程序本身。候选方案通过闭环执行进行选择，且在搜索结束后，若修复方案能够泛化到不同的环境变化和任务中，则将其有效加入技能库。当某个候选方案解决调试配置问题，或搜索预算耗尽时，搜索终止。算法 1 总结了演化搜索过程。流水线的详细信息见附录 E.4。

**Algorithm 1** Evolutionary search over programs

**Require:** task $\tau$, program $P^0$, sets $S_{\text{dbg}}, S_{\text{val}}$, skill library $\mathcal{L}$, agent $M$, budget $(K, T)$, threshold $\theta$
**Notation:** $\textsc{Execute}(P, S) = (r, Z)$ returns score $r$ and trace bundle $Z$.
$(r^\star, Z^0) \leftarrow \textsc{Execute}(P^0, S_{\text{dbg}})$; $P^\star \leftarrow P^0$
$\mathcal{H} \leftarrow \{(P^0, r^\star, Z^0)\}$
**for** $i = 1, \ldots, T$ **do**
  $\{P_i^k\}_{k=1}^K \leftarrow$ $\textsc{ProposeRepairs}(M, \tau, \text{Top3}(\mathcal{H}), \mathcal{L}, \mathcal{H})$
  **for** $k = 1, \ldots, K$ **do**
    $(r_i^k, Z_i^k) \leftarrow \textsc{Execute}(P_i^k, S_{\text{dbg}})$
  **end for**
  $\mathcal{H} \leftarrow \mathcal{H} \cup \{(P_i^k, r_i^k, Z_i^k)\}_{k=1}^K$; $k^\star \leftarrow \arg\max_k r_i^k$
  **if** $r_i^{k^\star} > r^\star$ **then**
    $(P^\star, r^\star) \leftarrow (P_i^{k^\star}, r_i^{k^\star})$
  **end if**
  **if** $r^\star \geq \theta$ **then**
    **break**
  **end if**
**end for**
$(r_{\text{val}}, Z_{\text{val}}) \leftarrow \textsc{Execute}(P^\star, S_{\text{val}})$
$\mathcal{G} \leftarrow$ $\textsc{ExtractValidatedPatterns}(\mathcal{H}, P^\star, r_{\text{val}}, Z_{\text{val}})$
**return** $(P^\star, r_{\text{val}}, \mathcal{G})$

## 3. 实验

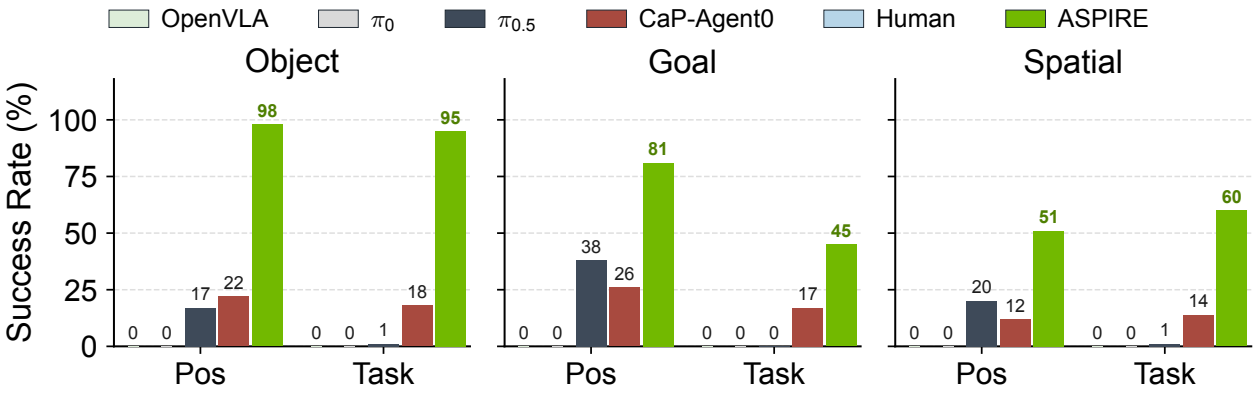
### 3.1. 实验设置

在所有的仿真基准测试中，我们使用 Claude Code [Anthropic, 2025] 搭配 Claude Opus 4.6 和 100 万 token 的上下文窗口作为编码智能体。该智能体在 CaP-X [Fu et al., 2026] 上编写可执行的 Python 机器人程序，CaP-X 是一个基于 MuJoCo Playground [Zakka et al., 2025] 构建的开源代码即策略框架，提供感知、几何和运动规划的机器人编程 API。所有实验中，智能体、环境和 API 集合均保持不变。

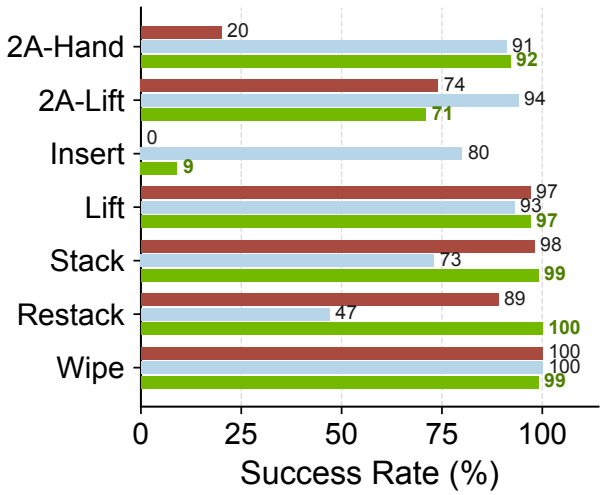
对于真实机器人技能迁移研究（§3.6），我们在双臂 YAM 操作站上使用 OpenAI Codex GPT-5.5 的 reasoning-xhigh 模式。我们选取了ASPIRE 在 Franka-based 仿真中编译的三种技能：汽水罐拾取、碗放置于盘子上、抽屉推拉。我们将这些作为上下文引导提供给真实机器人编程智能体。真实机器人情景采用与仿真不同的实体形态和 API，但暴露了ASPIRE 的多模态执行迹，使得智能体能够在调试环中自主运行、检查和调试程序，而无需任务特定的人工指导。我们对比了有无相应仿真发现技能的运行情况，测量首次成功所需的 token 数以及保留的真实机器人成功率。

### 3.2. 基准和基准

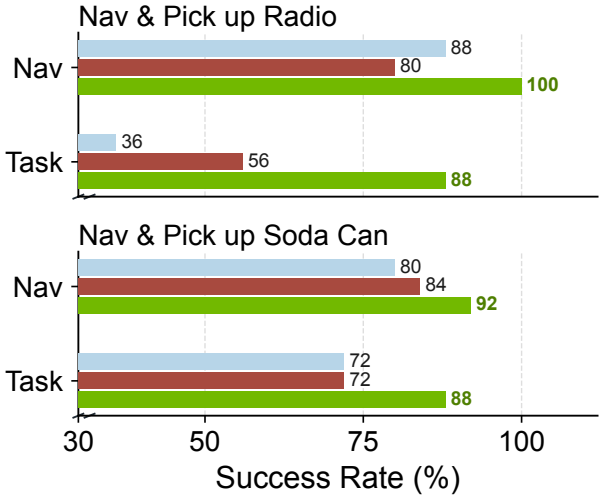
我们在三个基准测试家族上评估 ASPIRE ：LIBERO-Pro [Zhou et al., 2025]，用于在物体、目标和空间扰动下评估短视界鲁棒性；Robosuite [Zhu et al., 2020]，用于接触丰富的单臂与双臂操作；以及 BEHAVIOR-1K [Li et al., 2024]，用于在 NAVIGATE-AND-PICK-UP-SODA-CAN 和 NAVIGATE-AND-




(a) **LIBERO-Pro** [Zhou et al., 2025]：在 10 个任务上的宏平均成功率 × 每个套件/扰动使用 50 个预留的环境种子。对于每个任务，ASPIRE 在种子 51–65 上学习并收集技能，并在种子 1–50 上评估一个生成的程序。




(b) **Robosuite** [Zhu et al., 2020]：每个任务在 100 次保留试验中成功率。ASPIRE 在种子 101–125 上进行学习，并在种子 1–100 上对每个任务评估一个生成的程序；2A 任务为双臂任务。




(c) **BEHAVIOR-1K** [Li et al., 2024]：在两个家用任务上的长时程移动操作。ASPIRE 在种子 26–35 上进行学习，并在种子 1–25 上进行评估，采用增量块执行；分别报告导航成功率和任务成功率。

Figure 4: **Aspire 在三个基准系列上优于先前的编码智能体和端到端视觉-语言智能体。**
(a) LIBERO-Pro 上的短时程操作；(b) Robosuite 上的高接触操作；(c) BEHAVIOR-1K 上的长时程移动操作。ASPIRE 在保留的种子上对每个任务评估一个生成的程序，而 CaP-Agent0 则在测试时通过推理和重试为每个种子重新生成一个分离的程序。ASPIRE 在所有基准上均提升了性能，部分结果甚至超过了人类专家编写的程序。详细的逐任务结果见附录 B 和 D.1。

PICK-UP-RADIO 任务上评估长视界家庭移动操作。我们的主要编码智能体基准是 CaP-Agent0 [Fu et al., 2026]，它采用视觉差分、预定义技能库以及每回合测试时重试机制。我们还与端到端视觉-语言-动作（VLA）策略进行了对比，包括 OpenVLA [Kim et al., 2024]、$\pi_0$ [Black et al., 2025] 和 $\pi_{0.5}$ [Physical Intelligence et al., 2025]。对于 zero-shot 迁移，我们在 LIBERO-90 [Liu et al., 2023] 上累积的技能库基础上，评估未参与训练的 LIBERO-Pro Long 任务。

### 3.3. 评估协议

在所有基准测试中，环境随机种子固定了每个任务实例，包括物体姿态、干扰项以及机器人/物体的初始状态。我们使用不相交的调试和评估随机种子：ASPIRE 在小规模的调试数据集上进行学习，然后在更大规模的保留评估随机种子上报告成功率，每个 LIBERO-Pro/Robosuite 任务生成一个程序；而 CaP-Agent0 则在测试时通过推理和重试为每个随机种子重新生成独立的程序。在 BEHAVIOR-1K 评估中，ASPIRE 采用增量块执行方式，从当前多模态迹中生成下一个代码块。

### 3.4. 主要评估结果

Figure 4 总结了主要评估结果。其中，“Human” 表示由人类专家编写的程序。在 LIBERO-Pro 上，ASPIRE 在所有三个套件中均优于所有基准：平均位置扰动和任务扰动轴，ASPIRE 在 Object 套件上比最强基线提升 77%，在 Goal 套件上提升 41.5%，在 Spatial 套件上提升 42.5%。$\pi_{0.5}$ 在某些位置扰动上强于 OpenVLA 和 $\pi_0$，但仍远低于ASPIRE ，并且在任务改写下基本崩溃。在 Robosuite 上，ASPIRE 在较简单的接触密集型任务上保持了接近饱和的性能，并将双手交接任务从 20% 大幅提升至 92%。在 BEHAVIOR-1K 上，ASPIRE 在导航和任务成功率上均优于人类专家和 CaP-Agent0，其中在NAVIGATE-AND-PICK-UP-RADIO任务上相比 CaP-Agent0 取得了最大任务级提升，从 56% 提升至 88%。

### 3.5. 零样本迁移至未见任务

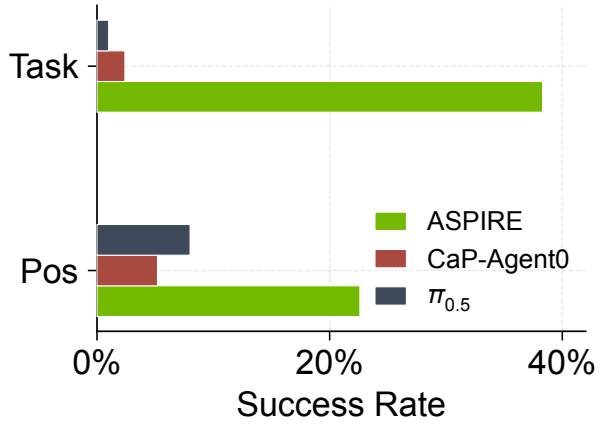
我们评估在 LIBERO-90 上积累的修复技能是否能够 zero-shot 传递到未见的 LIBERO-Pro Long 任务。ASPIRE 从 $N \in \{0, 25, 50, 90\}$ 源任务构建库快照，其中 $N$=0 为无库情景，$N$=90 为完整套件。对于每个未见任务，ASPIRE 生成一个程序，并在不进行额外调试、重试或任务特定库更新的情况下，跨种子评估该程序。

图 5(a) 将完整的 $N$=90 技能库与先验基准方法在两个 LIBERO-Pro Long 转移轴上进行了比较：ASPIRE 在位置扰动下的成功率为 23%，在任务扰动下的成功率为 38%，在两个轴上均优于 CaP-Agent0 和 $\pi_{0.5}$。图 5(b) 表明，随着技能库大小的增长，成功率持续提升，这表明来自短时程任务的有效修复为长时程组合提供了可复用的机器人知识。

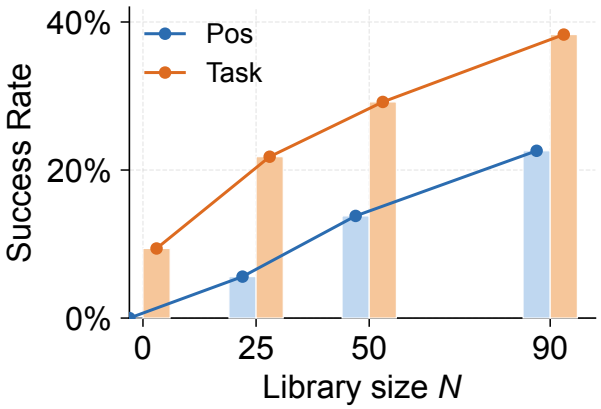
### 3.6. 真实机器人在不同身体形态间的技能迁移

我们评估在仿真中习得的技能是否能够减少在具身形态不同的真实机器人上进行调试的工作量。这种迁移并非直接部署策略：真实机器人使用其自身的感知、校准和控制栈，编码智能体仍需通过现实世界执行反馈来调整程序。问题是，检索在仿真中发现的技能能否提供有用的上下文引导，从而减少达到成功程序所需的现实世界调试量。

表 1 显示，迁移技能始终能降低调试成本，但其对最终成功率的影响取决于任务。碗放置在两种情景下均成功，但在使用技能检索时所用 token 更少；汽水罐拾取的成功率从 13/20 提升至 19/20，同时总 token 数减少了近一个数量级；抽屉操作在技能引导下达到 11/20 的成功率，而无技能基准




(a) 完整 $N$=90 库 zero-shot 结果与基准对比。




(b) 随着技能库大小的增加，zero-shot 成功率得到提升。

Figure 5: **LIBERO-Pro Long 上的跨任务 zero-shot 迁移**。在 LIBERO-90 上积累的技能提升了在未见的长时程任务上的 zero-shot 性能。图 (a) 将完整的 $N$=90 库与基准方法进行了比较。图 (b) 展示了随着技能库大小增加，位置/任务成功率的变化情况。所有成功率均在每轴 10 个任务上进行宏平均。各任务的具体结果见附录 C。

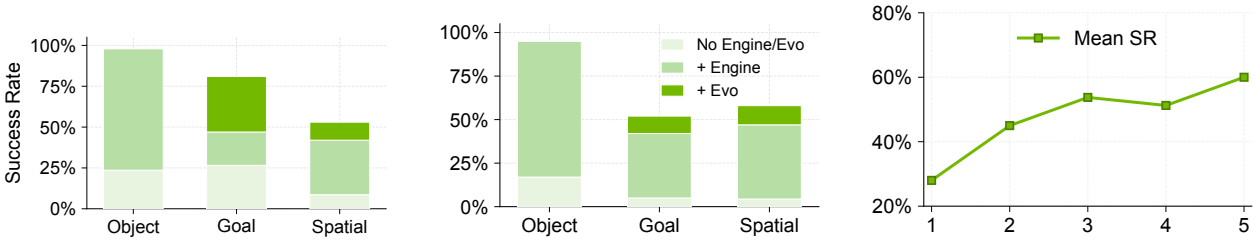
Table 1: **真实机器人跨机体技能迁移**。对于每个任务，我们对比了在真实机器人上进行调试时，是否从ASPIRE的库中检索对应仿真发现的技能。token 数量在首次成功的真实机器人程序出现时停止测量，或者在未达到成功的运行中耗尽调试预算时停止。成功率报告了生成程序的评估试验结果。

| Task | Output Tokens (M) ↓ | | Total Tokens (M) ↓ | | Success Rate ↑ | |
|---|---|---|---|---|---|---|
| | w/o Skills | w/ Skills | w/o Skills | w/ Skills | w/o Skills | w/ Skills |
| Put bowl on plate | 0.05 | 0.04 | 8.65 | 5.11 | 20/20 | 20/20 |
| Lift soda can | 0.18 | 0.03 | 61.94 | 6.58 | 13/20 | 19/20 |
| Open/push drawer | 1.33 | 0.36 | 334.917 | 81.67 | 0/20 | 11/20 |

则耗尽了更大的 token 预算却未能生成成功的评估程序。这些结果表明，所选的基于失败经验的技能能够指导跨实体化和 API 变化的真实机器人程序合成，而非仅仅记忆仿真器特定的代码。

### 3.7. 消融研究

**机器人执行引擎与演化搜索** 我们在 LIBERO-Pro 上对 ASPIRE 的两个关键组件进行了消融实验：机器人执行引擎，其暴露稠密的执行轨迹并验证任务级别的修复；以及演化搜索，其在机器人执行引擎调试后探索额外的修复候选。图 6(a) 和图 6(b) 将成功率分解为不包含任一组件的基础系统、加入机器人执行引擎带来的提升，以及演化搜索带来的额外提升。机器人执行引擎提供了最大的平均改进，使宏平均成功率从 14% 提升至 62%；而演化搜索进一步改善了剩余的难题，使两者结合时达到 72%。




(a) 正向扰动。 (b) 任务扰动。 (c) 每迭代进度。

Figure 6: **机器人执行引擎与演化搜索在 LIBERO-Pro 上的消融实验**。图 (a) 和图 (b) 展示了位置扰动和任务扰动的堆叠柱状图：基础系统（不包含机器人执行引擎或演化搜索）、加入机器人执行引擎带来的增益，以及演化搜索带来的额外增益。平均而言，机器人执行引擎提供的增益最大，将宏平均成功率从 14% 提升至 62%；演化搜索进一步改善了剩余的难题。图 (c) 展示了在演化搜索迭代过程中，低表现任务的平均进展。各任务的具体结果见附录 D。

**演化搜索迭代** 图 6(c) 跟踪了随着演化搜索预算增加，低表现任务上的性能变化。在前几轮中，成功率稳步提升，表明多次采样修复假设能够快速恢复单次迭代调试所遗漏的备选方案。之后曲线继续缓慢上升，表明额外的迭代仍有助于解决剩余的难题，但收益递减。

## 4. 相关工作

**Agentic 机器人控制**。 机器人控制已通过端到端的视觉-语言-动作策略以及可执行程序进行研究，这些程序组合了感知、规划和控制 API [Ahn et al., 2022, Brohan et al., 2023, Octo Model Team et al., 2024, Kim et al., 2024, Black et al., 2025, Mu et al., 2024, Physical Intelligence et al., 2025, NVIDIA et al., 2025, Sun et al., 2026, Li et al., 2026, Fu et al., 2026]。软件工程智能体为代码提供了相关的编写-执行-调试环 [Jimenez et al., 2024, Wang et al., 2025, Yang et al., 2024, Wang et al., 2024a, Chen et al., 2024, Anthropic, 2025, OpenAI, 2025, anomalyco, 2025]。ASPIRE 基于可执行程序范式，但专注于持续的具身改进：智能体自由选择原始级别的多模态迹，编写修复，验证，并在任务间保留成功的经验。

**自改进智能体与技能库**。 大模型智能体通过开放式的记忆、技能库以及自我演化的技能资源库得到了改进 [Wang et al., 2023, Tziafas and Kasaei, 2024, Peng et al., 2024, Fu et al., 2025, Zhang et al., 2026, Xie et al., 2026]。其他系统则利用大模型生成奖励、课程、环境或搜索候选方案 [Ma et al., 2024a, Yu et al., 2023, Xie et al., 2024, Liang et al., 2024, Ma et al., 2024b, Du et al., 2023, Wang et al., 2024b, Romera-Paredes et al., 2024, Novikov et al., 2025, Guo et al., 2025, 2026, Cao et al., 2026]。与仅记录成功经验、文本反思或用于下游策略训练的奖励函数不同，ASPIRE 存储的是从带有属性的具身失败中提取出的有效修复知识。其技能在内容和准入方面均具有开放性：它们源自智能体自身的调试经验，涵盖异构类别（§2.2），并作为上下文引导被复用于未来的程序修复。演化搜索（§2.3）进一步拓宽了可执行程序修复的范围，在可复用模式被纳入库之前进行探索。

## 5. 局限性

Aspire has several important limitations. 首先，尽管所选仿真发现的技能支持在不同实体上实现自主真实机器人调试，但我们的系统尚未成为完全自主的真实世界终身学习器。与仿真环境中的成功检测和场景重置成本低廉且可程序化不同，真实世界部署仍需具备鲁棒的成功检测、安全重置、安全监控以及校准维护能力。未来工作应完善这一评估-重置环，以实现更广泛真实世界任务集上的仿真到现实的技能迁移。

其次，我们的方法依赖于一个冻结的前沿大模型（Claude Opus 4.6，具有 100 万 token 的上下文窗口）来解释多模态迹、编写程序修复并提出演化搜索候选；我们尚未验证较小或较弱的大模型是否能维持相同的调试环。

第三，Aspire 使用预定义的感知、规划和控制原语 API 来编写程序。该 API 使调试变得易处理且安全，但也限制了智能体所能表达的行为：若某任务需要超出暴露原语的感知、控制或交互能力，智能体要么低效地近似这些能力，要么依赖人类扩展 API。未来系统应研究智能体如何安全地提出、验证并整合新的机器人原语。

第四，当前技能库优先考虑已验证的可复用修复，但并未完全解决长期记忆管理问题。随着库的增长，部分条目可能变得过时、过于具体、冗余或对新任务产生误导，这可以解释 zero-shot 迁移中非单调趋势的存在。需要更稳健的检索、剪枝、秩排序和再验证机制，以确保库在大规模下依然有效。

最后，调试与演化搜索环计算开销较大，每个任务消耗大量大模型调用及模拟器或机器人轨迹回放；要扩展至超大规模任务集，将需要更廉价的大模型推理、更高样本效率的搜索，或更强的先前修复复用机制。

## 6. 结论

我们提出 Aspire ，一个持续学习的机器人系统，能够自主编写并优化机器人控制程序，同时将经验累积至可复用的技能库中。Aspire 在一个开放式的学习环中运行，包含三个组成部分：一个封闭环路的机器人执行引擎，可暴露细粒度的多模态迹；一个持续扩展的技能库，将经过验证的修复方案提炼为可迁移的知识；以及一种演化搜索过程，用于探索多样化的任务序列和控制程序。在多个基准测试中，Aspire 显著优于现有的 VLA 和编码智能体基准，展现出强大的 zero-shot 跨越未见长时程任务的迁移能力，并提供了初步证据表明，在仿真中发现的技能能够跨实体迁移，显著降低真实机器人编程的 token 成本，即使面对不同的机器人实体和 API 也是如此。

## 致谢

我们感谢 Nadun Ranawaka、Jimmy Wu、Matin Furutan、Haotian Lin、Abhi Maddukuri、Yulu Gan、Matin Nikoui 和 Yuqi Xie 在开源支持、真实机器人基础设施、论文建议与指导、网站发布以及实验设备方面提供的帮助。我们也衷心感谢 NVIDIA GEAR、UMich SymbioticLab、UsesysLab、

UC Berkeley AUTOLab 和 CMU LeCAR Lab 的成员们给予的支持。

## References

Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gober, Karol Hausman, et al. Do as i can, not as i say: Grounding language in robotic affordances. In *Conference on Robot Learning (CoRL)*, 2022.

anomalyco. Opencode: The open source ai coding agent, 2025. URL `https://github.com/anomalyco/opencode`. Accessed May 2026.

Anthropic. Claude code: Best practices for agentic coding, 2025. URL `https://www.anthropic.com/engineering/claude-code-best-practices`. Accessed April 2026.

Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. $\pi_0$: A vision-language-action flow model for general robot control. In *Robotics: Science and Systems (RSS)*, 2025.

Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning (CoRL)*, 2023.

Shiyi Cao, Ziming Mao, Joseph Gonzalez, and Ion Stoica. K-search: LLM kernel generation via co-evolving intrinsic world model. *arXiv preprint arXiv:2602.19128*, 2026.

Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *International Conference on Learning Representations (ICLR)*, 2024.

Yuqing Du, Olivia Watkins, Zihan Wang, Cédric Colas, Trevor Darrell, Pieter Abbeel, Abhishek Gupta, and Jacob Andreas. Guiding pretraining in reinforcement learning with large language models. In *International Conference on Machine Learning (ICML)*, 2023.

Honghao Fu, Junlong Ren, Qi Chai, Deheng Ye, Yujun Cai, and Hao Wang. Vistawise: Building cost-effective agent with cross-modal knowledge graph for minecraft. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025.

Max Fu, Justin Yu, Karim El-Refai, Ethan Kou, Haoru Xue, Huang Huang, Wenli Xiao, Guanzhi Wang, Fei-Fei Li, Guanya Shi, Jiajun Wu, Shankar Sastry, Yuke Zhu, Ken Goldberg, and Linxi Fan. Cap-x: A framework for benchmarking and improving coding agents for robot manipulation. *arXiv preprint arXiv:2603.22435*, 2026.

Ping Guo, Chenyu Zhu, Siyuan Chen, Fei Liu, Xi Lin, Zhichao Lu, and Qingfu Zhang. Evoengineer: Mastering automated CUDA kernel code evolution with large language models. *arXiv preprint arXiv:2510.03760*, 2025.

Ping Guo, Chao Li, Yinglan Feng, and Chaoning Zhang. Code evolution for control: Synthesizing policies via LLM-driven evolutionary search. *arXiv preprint arXiv:2601.06845*, 2026.

Wenlong Huang, Chen Wang, Ruohan Zhang, Yunzhu Li, Jiajun Wu, and Li Fei-Fei. Voxposer: Composable 3D value maps for robotic manipulation with language models. In *Conference on Robot Learning (CoRL)*, 2023.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. Openvla: An open-source vision-language-action model. In *Conference on Robot Learning (CoRL)*, 2024.

Chengshu Li, Ruohan Zhang, Josiah Wong, Cem Gokmen, Sanjana Srivastava, Roberto Martín-Martín, Chen Wang, Gabrael Levine, Wensi Ai, Benjamin Martinez, et al. Behavior-1k: A human-centered, embodied ai benchmark with 1,000 everyday activities and realistic simulation. In *Conference on Robot Learning (CoRL)*, 2024.

Ruiying Li, Yunlang Zhou, YuYao Zhu, Kylin Chen, Jingyuan Wang, Sukai Wang, Kongtao Hu, Minhui Yu, Bowen Jiang, Zhan Su, Jiayao Ma, Xin He, Yongjian Shen, Yang Yang, Guanghui Ren, Maoqing Yao, Wenhao Wang, and Yao Mu. Roboclaw: An agentic framework for scalable long-horizon robotic tasks. *arXiv preprint arXiv:2603.11558*, 2026.

Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023.

William Liang, Sam Wang, Hung-Ju Wang, Osbert Bastani, Dinesh Jayaraman, and Yecheng Jason Ma. Eurekaverse: Environment curriculum generation via large language models. In *Conference on Robot Learning (CoRL)*, 2024.

Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models. In *International Conference on Learning Representations (ICLR)*, 2024a.

Yecheng Jason Ma, William Liang, Hung-Ju Wang, Sam Wang, Yuke Zhu, Linxi Fan, Osbert Bastani, and Dinesh Jayaraman. Dreureka: Language model guided sim-to-real transfer. In *Robotics: Science and Systems (RSS)*, 2024b.

Yao Mu, Junting Chen, Qinglong Zhang, Shoufa Chen, Qiaojun Yu, Chongjian Ge, Runjian Chen, Zhixuan Liang, Mengkang Hu, Chaofan Tao, Peize Sun, Haibao Yu, Chao Yang, Wenqi Shao, Wenhai Wang, Jifeng Dai, Yu Qiao, Mingyu Ding, and Ping Luo. Robocodex: Multimodal code generation for robotic behavior synthesis. In *International Conference on Machine Learning (ICML)*, 2024.

Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, et al. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.

NVIDIA, Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, Joel Jang, Zhenyu Jiang, Jan Kautz, Kaushil Kundalia, Lawrence Lao, Zhiqi Li, Zongyu Lin, Kevin Lin, Guilin Liu, Edith Llontop, Loic Magne, Ajay Mandlekar, Avnish Narayan, Soroush Nasiriany, Scott Reed, You Liang Tan, Guanzhi Wang, Zu Wang, Jing Wang, Qi Wang, Jiannan Xiang, Yuqi Xie, Yinzhen Xu, Zhenjia Xu, Seonghyeon Ye, Zhiding Yu, Ao Zhang, Hao Zhang, Yizhou Zhao, Ruijie Zheng, and Yuke Zhu. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.

Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, Jianlan Luo, You Liang Tan, Lawrence Yunliang Chen, Pannag Sanketi, Quan Vuong, Ted Xiao, Dorsa Sadigh, Chelsea Finn, and Sergey Levine. Octo: An open-source generalist robot policy. In *Robotics: Science and Systems (RSS)*, 2024.

OpenAI. Codex: Ai coding partner from openai, 2025. URL `https://openai.com/codex/`. Accessed May 2026.

Shaohui Peng, Xing Hu, Qi Yi, Rui Zhang, Jiaming Guo, Di Huang, Zikang Tian, Ruizhi Chen, Zidong Du, Qi Guo, Yunji Chen, and Ling Li. Hypothesis, verification, and induction: grounding large language models with self-driven skill learning. In *Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence and Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence and Fourteenth Symposium on Educational Advances in Artifi-*

*cial Intelligence*, AAAI'24/IAAI'24/EAAI'24. AAAI Press, 2024. ISBN 978-1-57735-887-9. doi: 10.1609/aaai.v38i13.29376. URL https://doi.org/10.1609/aaai.v38i13.29376.

Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In *Conference on Robot Learning (CoRL)*, 2025.

Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.

Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. Progprompt: Generating situated robot task plans using large language models. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023.

Xiaoquan Sun, Zetian Xu, Chen Cao, Zonghe Liu, Yihan Sun, Jingrui Pang, Ruijian Zhang, Zhen Yang, Kang Pang, Dingxin He, Mingqi Yuan, and Jiayu Chen. Atomvla: Scalable post-training for robotic manipulation via predictive latent world models. *arXiv preprint arXiv:2603.08519*, 2026.

Georgios Tziafas and Hamidreza Kasaei. Lifelong robot library learning: Bootstrapping composable and generalizable skills for embodied control with language models. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. In *International Conference on Machine Learning (ICML)*, 2024a.

Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. Openhands: An open platform for ai software developers as generalist agents. In *International Conference on Learning Representations (ICLR)*, 2025.

Yufei Wang, Zhou Xian, Feng Chen, Tsun-Hsuan Wang, Yian Wang, Zackory Erickson, David Held,

and Chuang Gan. Robogen: Towards unleashing infinite data for automated robot learning via generative simulation. In *International Conference on Machine Learning (ICML)*, 2024b.

Senwei Xie, Yuntian Zhang, Ruiping Wang, and Xilin Chen. Uni-skill: Building self-evolving skill repository for generalizable robotic manipulation. *arXiv preprint arXiv:2603.02623*, 2026.

Tianbao Xie, Siheng Zhao, Chen Henry Wu, Yitao Liu, Qian Luo, Victor Zhong, Yanchao Yang, and Tao Yu. Text2reward: Reward shaping with language models for reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2024.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Wenhao Yu, Nimrod Gileadi, Chuyuan Fu, Sean Kirmani, Kuang-Huei Lee, Montse Gonzalez Arenas, Hao-Tien Lewis Chiang, Tom Erez, Leonard Hasenclever, Jan Humplik, Brian Ichter, Ted Xiao, Peng Xu, Andy Zeng, Tingnan Zhang, Nicolas Heess, Dorsa Sadigh, Jie Tan, Yuval Tassa, and Fei Xia. Language to rewards for robotic skill synthesis. In *Conference on Robot Learning (CoRL)*, 2023.

Kevin Zakka, Baruch Tabanpour, Qiayuan Liao, Mustafa Haiderbhai, Samuel Holt, Jing Yuan Luo, Arthur Allshire, Erik Frey, Koushil Sreenath, Lueder A. Kahrs, Carmelo Sferrazza, Yuval Tassa, and Pieter Abbeel. Mujoco playground. *arXiv preprint arXiv:2502.08844*, 2025.

Ziao Zhang, Kou Shi, Shiting Huang, Avery Nie, Yu Zeng, Yiming Zhao, Zhen Fang, Qishen Su, Haibo Qiu, Wei Yang, Qingnan Ren, Shun Zou, Wenxuan Huang, Lin Chen, Zehui Chen, and Feng Zhao. Skillflow:benchmarking lifelong skill discovery and evolution for autonomous agents, 2026. URL `https://arxiv.org/abs/2604.17308`.

Xueyang Zhou, Yangming Xu, Guiyao Tie, Yongchao Chen, Guowen Zhang, Duanfeng Chu, Pan Zhou, and Lichao Sun. Libero-pro: Towards robust and fair evaluation of vision-language-action models beyond memorization. *arXiv preprint arXiv:2510.03827*, 2025.

Yuke Zhu, Josiah Wong, Ajay Mandlekar, Roberto Martín-Martín, Abhishek Joshi, Kevin Lin, Abhiram Maddukuri, Soroush Nasiriany, and Yifeng Zhu. robosuite: A modular simulation framework and benchmark for robot learning. *arXiv preprint arXiv:2009.12293*, 2020.

## A. 技能库详情

附录将图 3 中所示的代表性条目扩展为技能库分类体系。每个条目将列出：(i) 从触发性错误迹中提取的问题；(ii) 编码情境检索条件的适用时机守卫；(iii) 已验证的修复片段；以及 (iv) 产生该条目的原始任务。

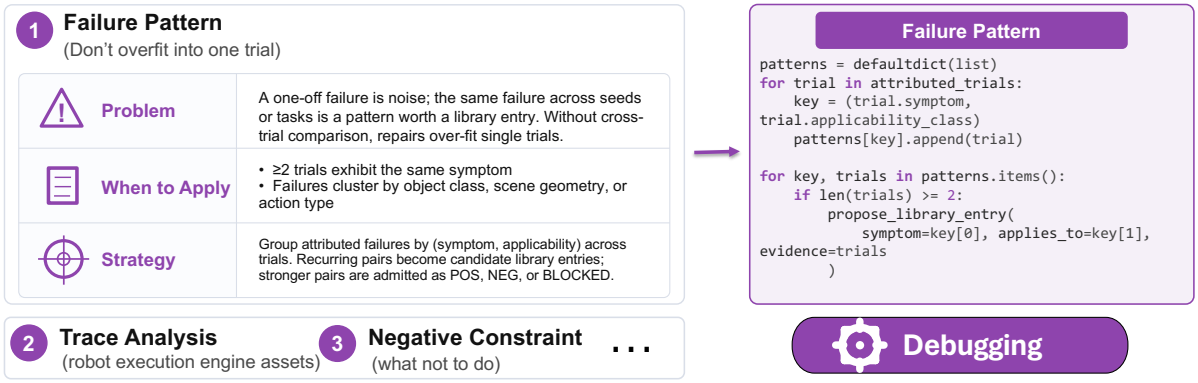



Figure 7: **调试技能**。代表性的技能库条目，编码了可重用的调试策略，包括失败特征、应用时机判断条件以及经过验证的修复草图。

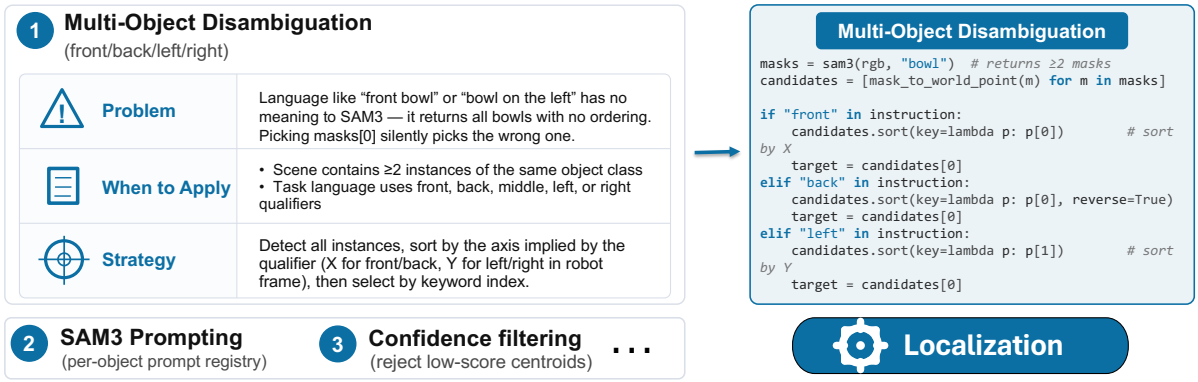



Figure 8: **定位技能**。将模糊语言和物体指代准确地定位到稳健的感知与定位流程中的代表性条目。

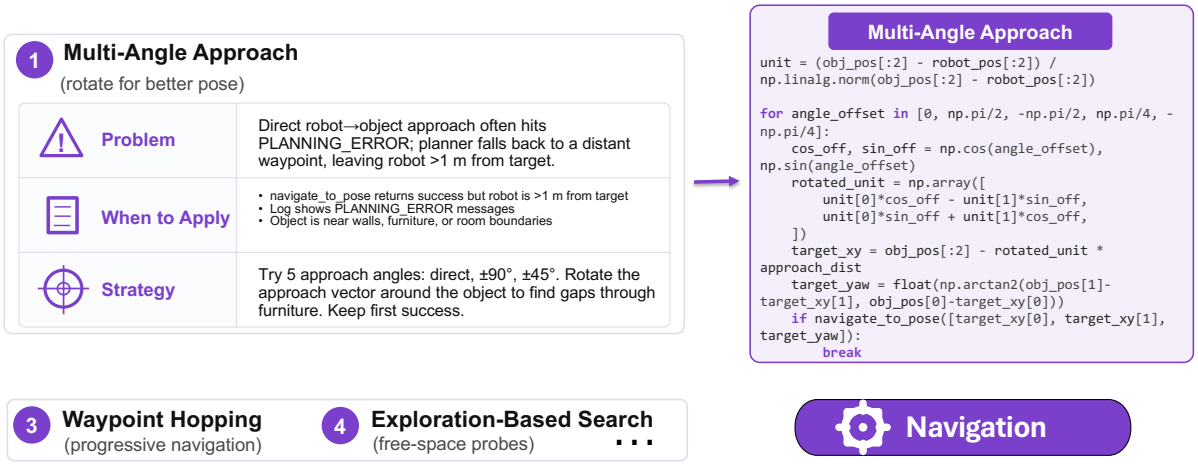



Figure 9: **导航技能**。用于从运动规划失败中恢复以及选择避障接近姿态的代表性条目。

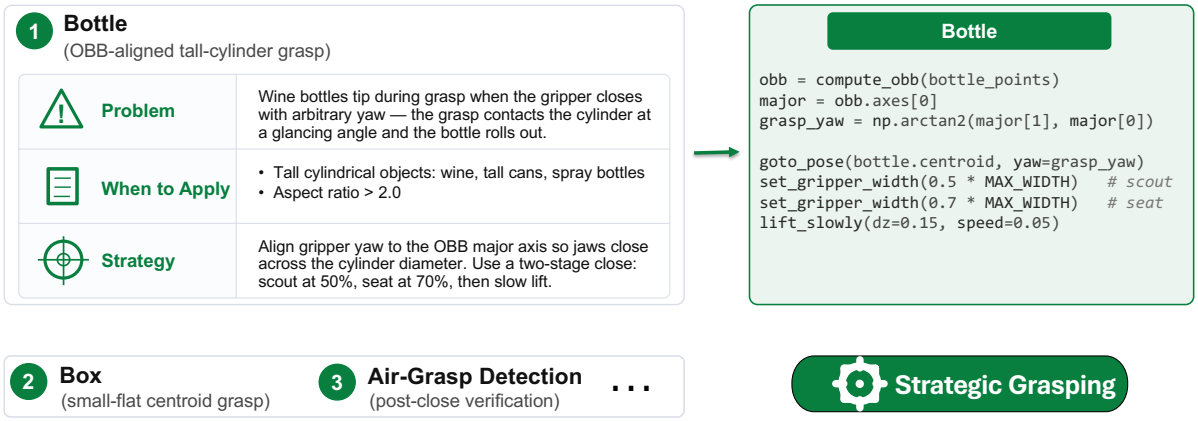



Figure 10: **策略性抓取技能**。选择任务合适的抓取点以及根据物体几何形状和场景上下文调整抓取策略的代表性实例。

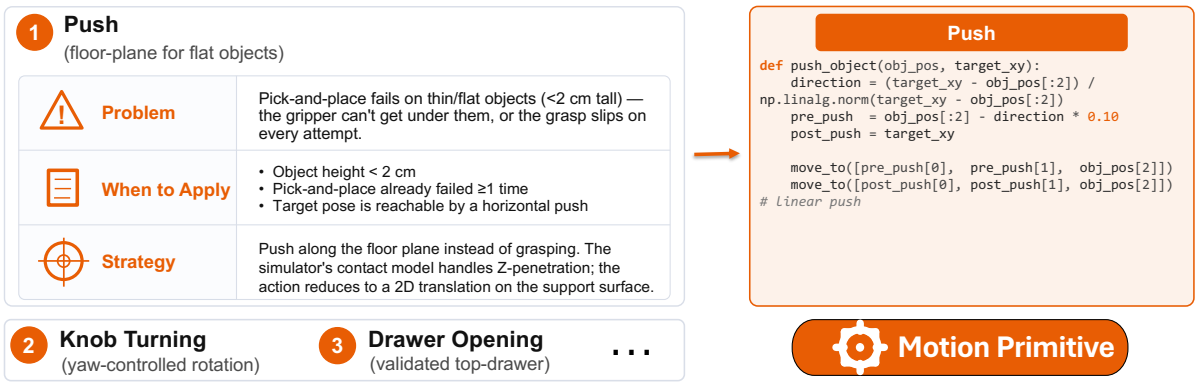



Figure 11: **运动基元技能**。可复用的低层级运动模式、接触丰富的对齐以及执行时恢复的代表性条目。

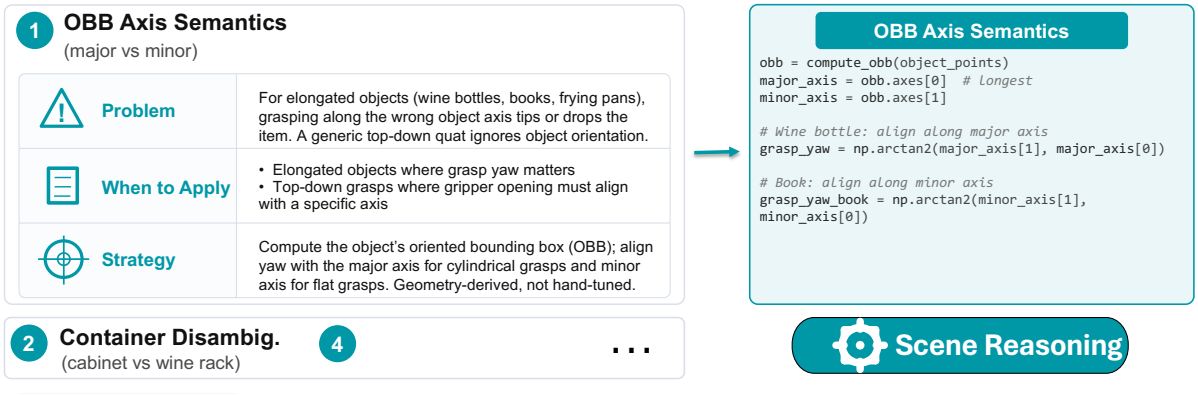



Figure 12: **场景推理能力**。代表性的条目包括对空间关系、支撑表面、遮挡以及场景级任务约束的推理。

# B. 主要基准结果表格

## B.1. 宏观平均主要结果

本节报告了主文中图 4 所展示的确切数值。

Table 2: LIBERO-Pro [Zhou et al., 2025] 在初始位置扰动（Pos）和指令扰动（Task）下，于 **libero-object**、**libero-goal** 和 **libero-spatial** 基准上的性能表现，以及 OpenVLA [Kim et al., 2024]、$\pi_0$ [Black et al., 2025]、$\pi_{0.5}$ [Physical Intelligence et al., 2025]、CaP-Agent0 [Fu et al., 2026] 与 ASPIRE 的对比结果，按每套基准 10 个任务进行宏平均。**总体**列为三个基准的宏平均结果。所有 ASPIRE 数值均基于 50 个种子的保留评估（种子 1–50）。各任务的详细分解见附录 D.1。

| **Method** | **libero-object** | | **libero-goal** | | **libero-spatial** | | **Overall** | | |
|---|---|---|---|---|---|---|---|---|---|
| | Pos | Task | Pos | Task | Pos | Task | Pos | Task | All |
| OpenVLA | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 |
| $\pi_0$ | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 |
| $\pi_{0.5}$ | 0.17 | 0.01 | 0.38 | 0.00 | 0.20 | 0.01 | 0.25 | 0.01 | 0.13 |
| CaP-Agent0 | 0.22 | 0.18 | 0.26 | 0.17 | 0.12 | 0.14 | 0.20 | 0.16 | 0.18 |
| **Aspire** | **0.98** | **0.95** | **0.81** | **0.45** | **0.51** | **0.60** | **0.77** | **0.67** | **0.72** |

Table 3: Robosuite [Zhu et al., 2020] 上 CaP-Agent0 [Fu et al., 2026] 和 ASPIRE 在 7 个操作任务上的表现（100 次试验的保留评估）。数值为任务成功率，单位为 $[0, 1]$。**平均**是所有 7 个任务上的未加权均值。

| **Task** | **CaP-Agent0** | **Aspire** |
|---|---|---|
| cube_lift | **0.97** | **0.97** |
| cube_stack | 0.98 | **0.99** |
| cube_restack | 0.89 | **1.00** |
| spill_wipe | **1.00** | 0.99 |
| two_arm_handover | 0.20 | **0.92** |
| two_arm_lift | **0.74** | 0.71 |
| nut_assembly | 0.00 | **0.09** |
| *Mean* | *0.68* | *0.81* |

Table 4: BEHAVIOR-1K [Li et al., 2024] 在两项代表性家庭任务上的表现：易拉罐拾取和收音机拾取。导航成功率和任务成功率在 $[0, 1]$ 中分别报告；所有数据均来自 25 组预留评估（种子编号 1–25）；ASPIRE 采用交互式逐块生成，其技能库基于种子编号 26–35 累积。

| **Task** | **Human** | | **CaP-Agent0** | | **Aspire** | |
|---|---|---|---|---|---|---|
| | Nav | Task | Nav | Task | Nav | Task |
| Soda Can pick-up | 0.80 | 0.72 | 0.84 | 0.72 | **0.92** | **0.88** |
| Radio pick-up | 0.88 | 0.36 | 0.80 | 0.56 | **1.00** | **0.88** |

# C. LIBERO-Pro 长距离 zero-shot 迁移

## C.1. 宏观平均主要结果

Table 5: ASPIRE 技能库在 LIBERO-90 上累积的宏平均 zero-shot 迁移至 LIBERO-Pro Long [Zhou et al., 2025]。$N$ 表示用于生成技能库的 LIBERO-90 任务数量；测试时不再进行额外调试。各列报告在 Pos（位置扰动）和 Task（指令扰动）轴上的宏平均成功率，每轴 10 个任务。

| **Method** | **LIBERO-Pro Long Zero-Shot Transfer** | | |
|---|---|---|---|
| | **Pos** | **Task** | **Overall** |
| OpenVLA | 0.00 | 0.00 | 0.00 |
| $\pi_0$ | 0.00 | 0.00 | 0.00 |
| $\pi_{0.5}$ | 0.08 | 0.01 | 0.05 |
| CaP-Agent0 | 0.052 | 0.024 | 0.038 |
| ASPIRE ($N$=0, zero-shot) | 0.00 | 0.094 | 0.047 |
| ASPIRE ($N$=25, zero-shot) | 0.056 | 0.218 | 0.137 |
| ASPIRE ($N$=50, zero-shot) | 0.138 | 0.292 | 0.215 |
| ASPIRE ($N$=90, zero-shot) | **0.226** | **0.383** | **0.305** |

## C.2. 按任务分解

Table 6: 将 ASPIRE 技能库（在 LIBERO-90 上累积）的 zero-shot 传输到 LIBERO-Pro Long，按任务和快照大小 $N$ （种子 1–50，$n$=50 ）。**Pos**：位置扰动变体。**Task**：语义扰动变体。

| | **Pos** | | | | **Task** | | | |
|---|---|---|---|---|---|---|---|---|
| **Task** | $N$=0 | $N$=25 | $N$=50 | $N$=90 | $N$=0 | $N$=25 | $N$=50 | $N$=90 |
| Stove + moka pot | 0.00 | 0.30 | 0.84 | **1.00** | 0.00 | 0.02 | 0.68 | 0.26 |
| Bowl in drawer | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 |
| Mug in microwave | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 |
| Both mokas on stove | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.84 | **1.00** |
| Soup + cream cheese | 0.00 | 0.12 | 0.02 | 0.00 | 0.00 | 0.68 | 0.60 | 0.70 |
| Soup + tomato sauce | 0.00 | 0.14 | 0.20 | 0.00 | 0.78 | 0.68 | 0.02 | 0.70 |
| Cream cheese + butter | 0.00 | 0.00 | 0.06 | **0.94** | 0.00 | 0.00 | 0.34 | 0.86 |
| Mug on two plates | 0.00 | 0.00 | 0.16 | 0.32 | 0.16 | 0.28 | 0.08 | 0.00 |
| Mug + pudding | 0.00 | 0.00 | 0.06 | 0.00 | 0.00 | 0.00 | 0.08 | 0.04 |
| Book in caddy | 0.00 | 0.00 | 0.04 | 0.00 | 0.00 | 0.52 | 0.28 | 0.26 |
| *Mean* | *0.00* | *0.06* | *0.14* | *0.23* | *0.09* | *0.22* | *0.29* | *0.38* |

# D. 消融细节

## D.1. LIBERO-Pro 任务扰动（交换）逐任务分解

Table 7: LIBERO-Pro 每任务消融实验，**位置扰动**（种子 1–50，$n$=50）。**Aspire 无机器人执行引擎和演化搜索**：zero-shot Claude Opus 4.6，使用 15 个示例程序。**Aspire 无演化搜索**：由机器人执行引擎修复的程序（执行引擎 + 技能库）。**演化搜索**：原始演化搜索结果（验证有效的最佳候选），若该任务未运行演化搜索则为 "–"。**Aspire**：在保留的验证集（种子 66–80）上，从修复程序与演化搜索结果中选出的每任务胜者。

| Task | w/o Engine & Evo | w/o Evo | Evo. search | Aspire |
|---|---|---|---|---|
| *libero-object* | | | | |
| Alphabet soup | 0.00 | **1.00** | – | **1.00** |
| BBQ sauce | 0.06 | **1.00** | – | **1.00** |
| Butter | 0.00 | **1.00** | – | **1.00** |
| Chocolate pudding | 0.00 | **1.00** | – | **1.00** |
| Cream cheese | 0.92 | **1.00** | – | **1.00** |
| Ketchup | 0.02 | **0.92** | – | **0.92** |
| Milk | 0.06 | **0.92** | – | **0.92** |
| Orange juice | 0.46 | **1.00** | – | **1.00** |
| Salad dressing | 0.18 | **0.96** | – | **0.96** |
| Tomato sauce | 0.66 | **0.98** | – | **0.98** |
| *Mean* | *0.24* | *0.98* | – | *0.98* |
| *libero-goal* | | | | |
| Open middle drawer | 0.28 | 0.02 | 0.72 | **0.72** |
| Open top drawer+bowl | 0.00 | **0.86** | – | **0.86** |
| Push plate to stove | 0.00 | 0.00 | 0.00 | 0.00 |
| Bowl on plate | 0.04 | 0.36 | 0.92 | **0.92** |
| Bowl on stove | 0.16 | 0.56 | 0.96 | **0.96** |
| Bowl on cabinet top | 0.72 | 0.74 | 1.00 | **1.00** |
| Cream cheese in bowl | 0.00 | **0.74** | 0.92 | **0.74** |
| Wine on rack | 0.06 | 0.46 | 0.98 | **0.98** |
| Wine on cabinet top | **1.00** | 0.24 | 1.00 | **1.00** |
| Turn on stove | 0.40 | 0.68 | 0.90 | **0.90** |
| *Mean* | *0.27* | *0.47* | $0.82^{(n=9)}$ | *0.81* |
| *libero-spatial* | | | | |
| Betw. plate+ramekin | 0.02 | 0.10 | 0.94 | **0.94** |
| From table center | 0.00 | **0.32** | 0.82 | **0.32** |
| In top drawer | 0.02 | **0.22** | 0.10 | 0.10 |
| Next to cookie box | 0.02 | **0.16** | 0.12 | **0.16** |
| Next to plate | 0.08 | **1.00** | 1.00 | **1.00** |
| Next to ramekin | 0.00 | **0.20** | 0.40 | **0.20** |
| On cookie box | 0.00 | **0.32** | 0.24 | 0.24 |
| On ramekin | 0.58 | 0.42 | 0.94 | **0.94** |
| On stove | 0.04 | **0.86** | – | **0.86** |
| On wooden cabinet | 0.10 | **0.56** | 0.34 | 0.34 |
| *Mean* | *0.09* | *0.42* | $0.54^{(n=9)}$ | *0.51* |
| *Overall mean* | *0.20* | *0.62* | $0.68^{(n=18)}$ | *0.77* |

## D.2. LIBERO-Pro 任务扰动（目标）按任务分解

Table 8: LIBERO-Pro 每任务消融实验，**任务扰动**（种子 1–50，$n$=50）。**Aspire 无机器人执行引擎和演化搜索**：zero-shot Claude Opus 4.6，包含 15 个示例程序。**Aspire 无演化搜索**：由机器人执行引擎修复的程序（执行引擎 + 技能库）。**演化搜索**：原始演化搜索结果（最佳验证候选），若该任务未运行演化搜索则为 "–"。**Aspire**：在保留的验证集（种子 66–80）上，从修复程序与演化搜索结果中选出的每任务胜者。

| Task | w/o Engine & Evo | w/o Evo | Evo. search | Aspire |
|---|---|---|---|---|
| *libero-object* | | | | |
| Alphabet soup | 0.00 | **1.00** | – | **1.00** |
| BBQ sauce | 0.88 | **1.00** | – | **1.00** |
| Butter | 0.78 | **0.98** | – | **0.98** |
| Chocolate pudding | 0.00 | **1.00** | – | **1.00** |
| Cream cheese | 0.00 | **0.72** | 0.74 | **0.72** |
| Ketchup | 0.00 | **0.96** | – | **0.96** |
| Milk | 0.00 | **1.00** | – | **1.00** |
| Orange juice | 0.00 | **1.00** | – | **1.00** |
| Salad dressing | 0.00 | **0.92** | – | **0.92** |
| Tomato sauce | 0.04 | **0.96** | – | **0.96** |
| *Mean* | *0.17* | *0.95* | – | *0.95* |
| *libero-goal* | | | | |
| Open middle drawer | 0.00 | 0.00 | 0.00 | 0.00 |
| Open top drawer+bowl | 0.00 | **0.50** | 0.46 | 0.46 |
| Push plate to stove | 0.26 | 0.58 | 0.76 | **0.76** |
| Bowl on plate | 0.00 | **0.88** | 0.82 | **0.88** |
| Bowl on stove | 0.00 | 0.00 | 0.00 | 0.00 |
| Bowl on cabinet top | 0.00 | 0.00 | 0.04 | **0.04** |
| Cream cheese in bowl | 0.00 | **0.82** | 0.74 | 0.74 |
| Wine on rack | 0.00 | 0.74 | 0.86 | **0.86** |
| Wine on cabinet top | 0.00 | **0.14** | 0.72 | **0.14** |
| Turn on stove | 0.24 | 0.52 | 0.66 | **0.66** |
| *Mean* | *0.05* | *0.42* | *0.51*$^{(n=10)}$ | *0.45* |
| *libero-spatial* | | | | |
| Betw. plate+ramekin | **0.14** | 0.08 | 0.84 | 0.08 |
| From table center | 0.26 | 0.46 | 0.64 | **0.64** |
| In top drawer | 0.04 | **0.96** | – | **0.96** |
| Next to cookie box | 0.00 | **0.94** | – | **0.94** |
| Next to plate | 0.08 | **0.70** | 0.92 | **0.70** |
| Next to ramekin | 0.00 | **0.60** | 0.42 | **0.60** |
| On cookie box | 0.00 | 0.06 | 0.70 | **0.70** |
| On ramekin | 0.20 | 0.10 | 0.46 | **0.46** |
| On stove | 0.00 | 0.32 | 0.42 | **0.42** |
| On wooden cabinet | 0.13 | **0.46** | 0.98 | **0.46** |
| *Mean* | *0.04* | *0.47* | *0.67*$^{(n=8)}$ | *0.60* |
| *Overall mean* | *0.09* | *0.61* | *0.59*$^{(n=19)}$ | *0.67* |

### D.3. 按任务划分的演化搜索进展

Table 9: 选定的 LIBERO-Pro 任务上的演化搜索进展。每一列表示在该迭代中表现最佳候选者的保留成功率（种子 1—50）。空白单元格表示搜索提前终止。

| Suite | Task | Iter 0 | Iter 1 | Iter 2 | Iter 3 | Iter 4 |
|---|---|---|---|---|---|---|
| goal_swap | Bowl → plate | 0.62 | 0.60 | 0.60 | 0.18 | **0.86** |
| goal_swap | Wine bottle → rack | 0.40 | **0.76** | 0.74 | | |
| goal_swap | Bowl → stove | 0.62 | **0.82** | | | |
| goal_task | Push plate → stove | 0.00 | **0.80** | | | |
| spatial_swap | Bowl on cabinet → plate | 0.16 | 0.38 | 0.38 | **0.46** | 0.30 |
| spatial_swap | Bowl next to cookie → plate | 0.16 | 0.20 | 0.04 | 0.04 | **0.40** |
| spatial_task | Bowl on cookie → plate | 0.26 | 0.04 | **0.70** | | |
| spatial_task | Bowl on ramekin → plate | 0.02 | 0.00 | 0.22 | **0.36** | 0.18 |

# E. 智能体流水线技能

## E.1. 技能库提示

演员智能体使用结构化的发现模式报告候选修复：观测到的故障模式、已验证的修复方案、可迁移的模式、任务特定的细节以及在调试配置上的验证成功率。协调器提示随后对每个候选修复进行可重用性审计，检查 API 策略合规性，并仅接纳那些很可能超出原始任务范围具有迁移潜力的已验证修复。并行的演员智能体被指示编写任务级别的修复与发现，而协调器则序列化技能准入以避免冲突的库写入操作。详见子智能体技能中的 E.3和 E.4中确切格式。所有智能体技能文件请参见代码发布。

## E.2. 智能体系统提示与持久记忆

CLAUDE.md

```
# CLAUDE.md
This file is the constitution for Claude Code working in this repository. Read it fully before doing
     anything.

## STARTUP: Read Project Memory First

At the start of every session, **immediately read** `.claude/memory/MEMORY.md` before doing anything else.
      It contains project vision, key conventions, and the skills index.

## What Is ASPIRE

ASPIRE is a framework where LLMs write Python code to control robots through a curated tool API. Code is
     executed in a sandboxed simulator, the simulator verifies success, and the agent improves through
     failure diagnosis and skill accumulation.

## FORBIDDEN APIs

**These APIs access simulator ground truth and are STRICTLY FORBIDDEN in all fix code, debug scripts, and
     skill implementations.** Using them invalidates benchmark results, as they don't transfer to real
     robots.

```
[FORBIDDEN] env.handle.env.sim - no MuJoCo sim object access
[FORBIDDEN] sim.data.body_xpos - no ground-truth object positions
    ... (rest of APIs omitted)
```
**Rule of thumb:** If a real robot with a camera could do it, it's allowed. If it reads the physics engine
     's internal state, it's forbidden.

**Also forbidden:** Reading simulator asset files (`.bddl`, `.xml`, `.urdf`, MuJoCo model files) to infer
     object geometry, success predicates, or scene structure. Treat these as inaccessible - diagnose
     purely from observations and traces.

## ALLOWED APIs
```

```
Full source: `integrations/franka/libero_reduced_skill_library.py` (and `libero_reduced.py` for base class
    ). Read the source for exact signatures, return types, and edge cases.
```
```
[ALLOWED] get_observation()                          - RGB, depth, intrinsics, extrinsics, robot state
[ALLOWED] segment_sam3_text_prompt(rgb, text)         - SAM3 text-prompted segmentation
    ... (rest of APIs omitted)
```
```
**You are encouraged to design new helper functions** (e.g. `localize_object()`, `make_topdown_quat()`)
    when you find yourself repeating patterns. Add them to the relevant skill in `.claude/skills/` so
    future sessions can reuse them.

## Skills

All skills live in `.claude/skills/<name>/SKILL.md` and are auto-discovered. Read the relevant skill
    before starting work.

| Skill | What it covers |
|---|---|
    ... (skills list omitted)
| **add more as needed** | **document new patterns here as you discover them** |

## Log Everything

Append to `./docs/logs/YYYY-MM-DD.md` after any significant work: experiments run, fixes tried, skills
    updated, results observed. Be detailed - include trial numbers, success rates, error messages, file
    paths.
```

MEMORY.md

```
# ASPIRE Project Memory

## What We're Building
A Self-Improving Coding Agent for robot manipulation. The agent debugs its own failures, accumulates
    reusable skills, and improves its perception tools.

- Foundation: Rich debugging playground (traces, REPL)
- Layer 1: Auto-growing skill library
- Layer 2: Program search (Pass@K parallel search)

## Key Conventions
- **NEVER git commit unless user explicitly asks**, do NOT auto-commit after skill updates or log writes
- **NEVER read sim asset files** (`.bddl`, `.xml`, `.urdf`, MuJoCo models), diagnose from observations and
     traces only
- **Always write reusable analysis as scripts** (not inline `python3 -c "..."`) and reference them in the
    relevant skill
    ... (rest of conventions omitted)

## Skills Available
    ... (skills list omitted)
```

api-reference.md

````
## Observation and Perception

| Function | Returns |
|---|---|
| `get_observation()` | RGB image, depth image, camera intrinsics, camera pose, and robot state. |
| `segment_text_prompt(rgb, text_prompt)` | A list of masks with bounding boxes, scores, and labels. |
| `segment_point_prompt(rgb, point_coords)` | A list of masks from point-conditioned segmentation. |
| `point_prompt_vlm(image, text_prompt)` | A pixel coordinate for the prompted object or region. |

## Grasp Planning and IK

| Function | Returns |
|---|---|
| `plan_grasp(depth, intrinsics, segmentation)` | Candidate grasp poses and scores in the camera frame. |
| `select_top_down_grasp(poses, scores, ...)` | The best top-down grasp transform and score. |
| `solve_ik(position, quaternion_wxyz)` | A 7-DoF joint configuration, or failure if unreachable. |

Camera-to-world conversion is required after grasp planning.

```python
camera_to_world = obs["agentview"]["pose_mat"]
grasp_world = camera_to_world @ grasp_camera
```

## Motion Execution

| Function | Returns |
|---|---|
| `move_to_joints(joints)` | Blocking joint-space motion. |
| `open_gripper()` / `close_gripper()` | Gripper command. |
| `goto_home_joint_position()` | Move to a predefined safe home configuration. |

## Geometry Utilities

| Function | Description |
|---|---|
| `decompose_transform(T)` | Convert a 4x4 transform into position and quaternion. |
| `mask_to_world_points(mask, depth, intrinsics, extrinsics)` | Project a 2D mask into a world-frame point
      cloud. |
    ... (rest omitted)

## Task Language

```python
env.task_language
```

The runtime task language is the reliable source of the actual goal, especially when benchmark filenames
     do not match remapped task instructions.

## Success Conditions
````

```
| Placement Type | Tolerance |
|---|---|
| Regular object placement | Tight XY tolerance plus contact. |
| Site-object placement | Larger site-local tolerance. |

## Scene Coordinate Frames

Use observation-derived depth projection for object positions and avoid hard-coded scene coordinates.
Reference frame constants are environment-specific and should not be used as object localization shortcuts
    .

`... (benchmark-specific table of scene constants omitted) ...`

## Output Directory Structure

```text
<OUTPUT_DIR>/
  <suite>/<task>/<run_id>/
    run/
      <done_flag>
      trial_<seed>_<attempt>_reward_<score>_taskcompleted_<flag>/
        code.py
        trace.json
        keyframes/
        video_combined.mp4
        visual_feedback.png
        summary.txt
        all_responses.json
```

The trial folder name records the seed, attempt index, reward, and task-completion flag.
The attempt index distinguishes clean exits from crashes, and `reward` is the scalar task score.

## Trace Format

```json
{
  "step": 5,
  "function": "segment_text_prompt",
  "args": {"rgb_shape": [512, 800, 3], "text_prompt": "bowl"},
  "duration_ms": 1463.8,
  "result": {
    "num_masks": 200,
    "mask_0_score": 0.917,
    "mask_0_bbox": [369.2, 257.1, 438.4, 310.5],
    "mask_0_area_pct": 0.68
  },
  "keyframe_saved": true
}
```
```

```
## Key Source Files

| File | Role |
|---|---|
| `<batch_runner>` | Runs suites, tasks, and model configurations. |
| `<trial_loop>` | Resets the environment, executes generated code, and saves artifacts. |
| `<single_trial_runner>` | Replays saved code or runs a fresh trial. |
| `<single_trial_analyzer>` | Performs structured trace analysis for one trial. |
| `<multi_candidate_trace_analyzer>` | Performs per-candidate trace analysis for evolutionary search runs.
    |
| `<progress_scanner>` | Aggregates success rates across an output directory. |
| `<robot_api_source>` | Defines the robot control and geometry API. |
| `<trace_logger_source>` | Records API calls, return values, timings, and keyframes. |
| `<evaluation_config>` | Defines benchmark and replay configuration. |
```

## E.3. ASPIRE 固定环技巧

coordinator.md

```
> **Purpose:** Coordinate repair of failed robot trials from a baseline policy and validate finalized
    fixes on a held-out seed set.
> **Baseline results:** `<BASELINE_OUTPUT_DIR>/`
> **Progress tracker:** `<PROGRESS_FILE>`
> **Subagent prompt:** `<SUBAGENT_PROMPT_TEMPLATE>`

### Initialization: Verify Perception Services

Before dispatching subagents, confirm that all required perception services are running. A service-
    specific health check should return the expected "up" status.

If any service is down, start it before proceeding. Subagents may fail silently without these dependencies
    .

### Main Loop

```text
read progress → assign free devices → dispatch subagents → go idle
                                                            ^
on notification: update progress/skills + redispatch device --|
```

One task is handled by one subagent on one compute device. Repeat until no eligible tasks remain.

### Coordinator Rules

1. **Dispatch subagents rather than debugging directly.** The coordinator assigns work, waits for
    completion notifications, updates shared knowledge, and dispatches the next task.
2. **Go idle after dispatching.** Completion notifications indicate when a device is free; avoid polling
    or intervening while subagents are running.
3. **Keep available devices occupied.** Any free device should receive the next pending task.
```

```
4. **Do not inspect task-specific debugging artifacts.** Files such as traces, generated programs,
     summaries, or keyframes are the subagent's responsibility. The coordinator may read the subagent's
     final findings when updating shared skills.
5. **Never re-dispatch completed tasks.** Completed validation tasks must not be run again, because
     duplicate trial directories can corrupt aggregate results.

### Workflow

#### 1. Read the Progress Tracker

```bash
cat <PROGRESS_FILE>
```

Identify tasks marked `pending` or `stage1-complete`.

#### 2. Check Free Compute Devices

Assign tasks only to devices that are free. Never assign two tasks to the same device.

#### 3. Dispatch One Subagent per Free Device

```python
Agent(
    description="Fix loop: <suite>/<task>",
    subagent_type="general-purpose",
    prompt=<filled_subagent_prompt>,
    run_in_background=True,
)
```

Send all dispatches together, then stop. The coordinator resumes when completion notifications arrive.

#### 4. On Completion: Redispatch and Update Skills

When a subagent completes, its result should include the device identifier. The coordinator then:

1. Reads the progress tracker.
2. Dispatches the next eligible task on the freed device.
3. Reads the completed task's `findings.md`.
4. Promotes generalizable patterns into the shared skill library.
5. Goes idle again.

If the result does not include a device identifier, query the device monitor to infer which device is free
     .

#### 5. Update Shared Skills

After each completion, read:

```text
<BASELINE_OUTPUT_DIR>/<SUITE>/<TASK>/findings.md
```
```

```
```

Promote only generalizable patterns to the skill library.

`... (skill-library category table omitted) ...`

### Validation Runs

For pure validation runs where final repair code already exists, direct scripts are more reliable than
    autonomous subagents. The validation script should run the held-out seed set exactly once and write
    to the official output directory.

`... (task-specific validation script, log parsing, and environment variables omitted) ...`

Important rule: pass only the validation base directory if the replay script appends `<suite>/<task>`
    internally.

### Suite Name Collision Warning

Some benchmark suites may contain identical task names under different perturbation conditions. Always
    include the full suite name when briefing a subagent.

### Perturbation Notes

- Position-perturbation suites randomize object poses by seed; perception should handle this naturally.
- Language-perturbation suites remap the task goal; the executable task
  instruction should be read from the runtime environment rather than inferred from a filename.
```

subagent.md

```
The following prompt is filled with one suite, task, and device assignment before being passed to a
    background subagent.

```text
## Task Assignment

SUITE: <SUITE_NAME>
TASK:  <TASK_NAME>
DEVICE: <DEVICE_ID>

Working directory: <PROJECT_ROOT>
```

### Evaluation-Set Lockout

The held-out validation seeds are locked during debugging.

- During Stage 1, replay only the debugging seeds.
- Do not replay validation seeds for testing, tuning, temporary experiments, or alternate code versions.
- Stage 2 uses the validation seeds exactly once, after `fix_code.py` is finalized, and writes to the
    official validation output directory.
```
```

````
- Stage 2 must run to completion before the subagent returns.

Violating this rule invalidates the benchmark result.

### Role

You are a task-level repair subagent for a robotics benchmark. Your job is to diagnose failed robot trials
     for one task, write a generalizable repair program, validate it according to the protocol, and
    report concise results.

### Context

The system asks language models to write Python control programs for a robot arm through a curated
    perception and manipulation API. Programs run in simulation, and failures are diagnosed through
    traces, summaries, and visual keyframes. The baseline policy has already run a set of debugging seeds
    ; your job is to repair the task program without using simulator ground truth.

#### Forbidden APIs

The repair program must not access simulator internals, ground-truth object states, hidden task predicates
    , asset files, or low-level physics state. Examples include:

```text
simulator body positions
site positions
`... (additional forbidden APIs omitted) ...`
```

These APIs would not transfer to a real robot and invalidate the result.

#### Allowed APIs

The repair code may use only observation-derived perception, manipulation primitives, and standard
    numerical computation:

```text
get_observation()
segment_text_prompt(rgb, text)
`... (additional allowed APIs omitted) ...`
```

Always use the project virtual environment or pinned runtime, not the system Python.

Before any replay, verify that the required perception services are running.

### Stage 1: Debug on Non-Held-Out Seeds

#### Step 0: Check Whether Repair Code Already Exists

If it exists, skip Stage 1 and check whether Stage 2 has already completed.

#### Step 1: Fast Path from Successful Baseline Trials
````

````
First, check whether any baseline debugging seed already succeeded:

If successful trials exist:

1. Read the successful generated program.
2. Extract the final code block if multiple attempts are present.
3. Test it on a small number of failed debugging seeds.
4. If it succeeds, save it as the task-level `fix_code.py` and proceed to Stage 2.
5. If it fails, continue to the full debug loop.

If no debugging seeds succeeded, continue directly to the full debug loop.

`... (exact replay command and temporary output paths omitted) ...`

#### Step 2: Diagnose Failed Debugging Seeds

For each failed seed, inspect the last attempt for that seed:

- `trace.json`: function calls, return values, perception failures, IK failures,
  and gripper signals.
- `code.py`: the baseline strategy.
- `keyframes/`: visual observations at perception steps.
- `summary.txt`: stdout, stderr, and reward.

Interpretation examples:

```text
large gripper width after closing → likely object grasped
medium gripper width → marginal grasp
small gripper width → likely air grasp
zero masks → poor perception prompt or occlusion
missing IK solution → target outside reachable workspace
early crash → inspect stdout/stderr summary
```

#### Step 3: Write and Test Repair Code

Before writing code, read the relevant skill-library entries for localization, grasping, transport, and
     manipulation patterns.

Write `fix.py` from the diagnosis and replay it on debugging seeds only.

If the repair fails, read the new trace and iterate. A live inspection REPL may be used for observation-
     derived checks such as segmentation counts.

`... (interactive REPL command and shell-specific buffering notes omitted) ...`

Hard limit: at most three replay attempts per debugging seed. Any run against a seed counts as an attempt,
      regardless of output directory or code version. After the cap, write a `BLOCKED.md` report and move
     on.
````

````
`BLOCKED.md` format:

`... (blocked-seed report template omitted) ...`

#### Step 4: Synthesize the Task-Level Repair Program

After debugging all seeds, synthesize one generalizable `fix_code.py`. The code must not contain seed-
    specific branches.

Save it to `<BASELINE_OUTPUT_DIR>/<SUITE>/<TASK>/fix_code.py`. Also write <BASELINE_OUTPUT_DIR>/<SUITE>/<
    TASK>/findings.md`.

Use this findings format:

```markdown
## Task: <SUITE> / <TASK>

### Root Cause(s)
- <concise description of each failure mode>

### What Fixed It
- <change that resolved each root cause>

### Perception Prompts That Worked
| Object | Prompts (priority order) | Notes |
|---|---|---|
| <object> | "<prompt1>", "<prompt2>" | <caveats> |

### Generalizable Patterns
- <patterns likely to apply beyond this task>

### Task-Specific Quirks
- <details specific to this task>

### Stage 2 Success Rate
<N>/<TOTAL_VALIDATION_SEEDS>
```

If all debugging seeds are blocked, skip Stage 2 for this task.

### Stage 2: Validate on Held-Out Seeds

Stage 2 is a one-shot validation operation. Do not debug Stage 2 runs.

Before running, check whether validation has already completed:

If enough unique validation seeds are already present, do not re-run Stage 2. Report the existing results
    and return. Otherwise, run validation sequentially:

`... (validation loop, log parsing, success counting, and environment variables omitted) ...`

Verify that outputs landed in the expected directory before reporting results.
````

```
### Final Action

Run the shared progress-generation script as the last action. It should update the progress tracker
    atomically and be safe under concurrent subagent execution.

### Return Format

Return only a brief structured summary:

```text
SUITE: <SUITE>
TASK: <TASK>
DEVICE: <DEVICE>

Stage 1: fix_code.py written: yes/no
  Seeds fixed: <count or list>
  Seeds blocked: <count and one-line cause>

Stage 2: <N>/<TOTAL_VALIDATION_SEEDS>
  Output dir: <VALIDATION_OUTPUT_BASE>/<SUITE>/<TASK>/

Key findings:
  - <generalizable root cause and fix>
  - <useful perception prompt, if notable>
  - <pattern that should inform future tasks>
```
```

## E.4. ASPIRE 演化搜索技能

演化搜索（第 2.3 节）维护一个持续的任务分析文档 $A_i$，该文档在各轮次间保持不变。$A_i$ 由三部分组成：(i) 从初始快照一次性填充的每项任务场景描述（物体形状、目标几何、障碍物、被阻塞的接近方向），(ii) 运行中的假设以及测试这些假设的候选元数据，以及 (iii) 一份记录了通过测试被排除的方向（ *eliminated*）和因工作空间约束而失败但尚未测试的方向（*blocked, untested*）的账本。UpdateAnalysis 每轮根据新的迹线、关键帧和库检索结果重写 $A_i$。正是这份文档使得后续轮次能够避免重复测试已被排除的分支，同时在新技巧（例如手腕旋转）可用时仍可重新尝试被阻塞的分支。

coordinator.md

```
> **What:** Run Evolutionary search iterative debugging with multiple candidates per iteration on a list
    of low-performing tasks.
> **Why:** Intensive debugging for difficult tasks; multi-candidate search can find strategies the single-
    pass actor misses.
> **Subagent template:** `<EVOSEARCH_SUBAGENT_PROMPT_TEMPLATE>`

### Initialization: Verify Perception Services
```

````
Before dispatching subagents, verify that all required perception services are running.
All services must be up before dispatching.

`... (service health-check command and machine-specific ports omitted) ...`

### Directory Layout

```text
<EVOSEARCH_OUTPUT_DIR>/
  <suite>/<task>/<run_id>/
    task_analysis.md
    iter_00/ iter_01/ ...
      candidate_A/code.py
      iter_summary.json
  <suite>/<task>/
    evosearch_best_code.py
    findings.md

<VALIDATION_OUTPUT_DIR>/
  <suite>/<task>/... (rest omitted)
```

### Task List

Run `<progress_generation_script>` to verify exact task names before dispatch.
Tasks below the chosen baseline success threshold are candidates for repair.

`... (experiment-specific task ordering and threshold table omitted) ...`

### The Loop

```text
read progress → assign free devices → dispatch subagents → go idle
                                                           ^
on notification: redispatch freed device to next pending task -|
```

One task = one subagent = one compute device.
The subagent runs iterations on the debug set and then runs Stage 2 on the held-out set before returning.
When the coordinator gets a completion notification, the device is already free; dispatch the next task.

### Coordinator Rules

1. **Dispatch subagents -- never run iterations yourself.**
2. **Go idle after dispatching.** You will be notified when a subagent finishes.
3. **Keep available devices occupied.**
4. **On each notification: check which device freed up, dispatch next pending task to it.**
5. **Never re-dispatch a done task.** `done` means Stage 2 complete with enough unique held-out seeds on
     disk.

### Workflow
````

````
#### 1. Check Progress

Check which tasks have a saved `evosearch_best_code.py` and how many held-out seeds have already been
     evaluated.

`... (progress-check shell loop omitted) ...`

#### 2. Check Free Compute Devices

Assign tasks only to devices that show no active jobs.

`... (device-monitoring command omitted) ...`

#### 3. Dispatch Subagents

Use the stronger reasoning model selected for the experiment.

```python
Agent(
    description="Evolutionary search actor: <suite_short>/<task_short> device<N>",
    subagent_type="general-purpose",
    model="<HIGH_CAPACITY_MODEL>",
    prompt=<filled_evosearch_subagent_prompt>,
    run_in_background=True,
)
```

Send all dispatches in one message, one per free device, then stop.

#### 4. On Each Completion: Redispatch

When a subagent notification arrives, it has already completed both Stage 1 and Stage 2.
Read `<EVOSEARCH_OUTPUT_DIR>/<SUITE>/<TASK>/findings.md`, note Stage 1 and Stage 2 rates, check free
     devices, dispatch the next pending task, and go idle.

### Stopping Criteria Reference

Subagents stop Stage 1 when the best candidate reaches the success threshold on debug seeds, the maximum
     number of iterations is reached, or the task is blocked.
If Stage 1 is blocked, the subagent skips Stage 2 and returns immediately.
````

subagent.md

````
The following prompt is filled with one suite, task, device assignment, baseline rate, and output location
      before dispatch.

```text
## Task Assignment

SUITE: <SUITE>
````
````

```
TASK: <TASK>
DEVICE: <DEVICE_ID>
TASKSHORT: <SHORT_UNIQUE_NAME_FOR_LOGS>
BASELINE_RATE: <BASELINE_RATE>
EVOSEARCH_DIR: <EVOSEARCH_OUTPUT_DIR>
EVALDIR: <VALIDATION_OUTPUT_DIR>

Working directory: <PROJECT_ROOT>
```

### Evaluation-Set Lockout During Iterations

The held-out evaluation seeds must not be run during iterations.
All iteration evaluations use only the debug seed set.
Stage 2 uses the held-out seeds after iterations converge and the final code is chosen.
Violation invalidates the benchmark.

### Role

You are a debugging subagent.
Your job is to run evolutionary search iterative debugging on debug seeds until convergence or plateau, save the best code, run Stage 2 on held-out seeds, and return a structured findings report with both Stage 1 and Stage 2 results.
Use the project virtual environment or pinned runtime, never the system Python.

### Context

The actor pipeline achieved `$BASELINE_RATE` on this task.
Your target is to exceed that significantly, ideally reaching the Stage 1 success threshold on debug seeds.
Read the skill library before writing candidates.
Verify that the required perception services are running before replay.

`... (experiment-specific config path and service-port checks omitted) ...`

### Forbidden APIs

```text
simulator body positions
site positions
`... (rest of forbidden APIs omitted)
```

### Stage 1: Evolutionary Search Iterations on Debug Seeds

#### Step 0: Check Whether Best Code Already Exists

If `evosearch_best_code.py` exists, skip to the return step.

#### Step 0b: Task-Language Remapping Check

The task name might not match the actual goal.

```
Always check and print the runtime task language.
If it differs from `$TASK`, the runtime task language is ground truth; base all strategy on it and record
     the actual language in `task_analysis.md`.

#### Step 1: Read Skills and Existing Baseline

Read skill-library entries for grasping, localization, transport, manipulation, and any task-specific
     companion skills.
Also read existing repair code if it exists.
If baseline code is found, record its path in `task_analysis.md` and seed it as `candidate_A` verbatim.

`... (skill-file reads and baseline fallback command omitted) ...`

#### Step 2: Create Run Directory and Scene Snapshot

Create a timestamped run directory under `<EVOSEARCH_OUTPUT_DIR>/<SUITE>/<TASK>/`.
Before writing candidates, run a scene snapshot on one debug seed and inspect the wide and wrist-view
     images.
Populate `task_analysis.md` with object shape, grasp strategy, goal geometry, placement strategy,
     hypotheses for the first iteration, and an iteration log.

`... (snapshot command and task_analysis.md template omitted) ...`

#### Step 3: Write K Candidates for the First Iteration

Always seed `candidate_A` from the existing baseline repair code if it exists.
Each candidate must test a distinct hypothesis.
No two candidates should fail at the same stage for the same reason.
Each candidate should include a docstring describing the hypothesis, how it differs from prior candidates,
      and the expected failure mode if wrong.

#### Step 4: Evaluate the Iteration

Evaluate all candidates on the fixed debug seed set.
Use the same debug seeds across all iterations so cross-iteration comparison is fair.
Read the leaderboard after completion and analyze traces.

`... (multi-candidate evaluation command, leaderboard parser, and trace analyzer command omitted) ...`

#### Step 5: Iterate

Stop Stage 1 when any stopping criterion is met.
When progress stalls, use the remaining iteration budget to explore structurally new approaches rather
     than stopping early.
A plateau on the current approach family means the current family may be wrong, not that the task is
     unsolvable.

The debug seeds are a small, potentially unrepresentative sample.
The code will be evaluated on unseen held-out seeds; write for those, not for the debug seeds.
Prefer strategies that work for mechanistic reasons over strategies that exploit patterns specific to
     debug-seed failures.
```

```
Avoid hard-coded thresholds, image-region masks, or offsets derived by fitting to observed debug-seed
     failures.

Per iteration:

1. Update `task_analysis.md` with new geometry from keyframes.
2. Append an iteration log with the leaderboard, eliminated hypotheses, and open questions.
3. Write K new candidates seeded from the top survivors of the current iteration.
4. Evaluate all candidates on the same debug seed set.
5. Read trace-analysis output and watch for arm blocking, gripper-width signals, perception failures, and
     IK failures.

#### Step 6: Save Best Code, Run Stage 2, Write Findings

Save the best final candidate as `<EVOSEARCH_OUTPUT_DIR>/<SUITE>/<TASK>/evosearch_best_code.py`.
If the best candidate does not beat the baseline candidate on debug seeds, fall back to the stronger
     baseline so Stage 2 uses the best available code.
If all approaches fail, mark the task blocked and skip Stage 2.

`... (best-candidate sanity check and copy commands omitted) ...`

### Stage 2: Evaluate Best Code on Held-Out Seeds

Run the selected best code on each held-out seed exactly once.
Do not use Stage 2 results to revise the code.
Poll until the validation loop completes, then read final success counts.

`... (detached validation script, polling commands, and final count commands omitted) ...`

### Findings Report

Write `<EVOSEARCH_OUTPUT_DIR>/<SUITE>/<TASK>/findings.md`:

```markdown
## Task: <SUITE> / <TASK>
## Baseline rate: <BASELINE_RATE>
## Best evosearch rate on debug seeds: <N>/<DEBUG_TOTAL> (<pct>
## Stage 2 on held-out seeds: <N>/<HELD_OUT_TOTAL> (<pct>

### Evolutionary Search Run
- Run dir: <RUN_DIR>
- Iterations completed: <N>
- Stopping reason: <solved|max_iterations|blocked>

### What Fixed It vs Baseline
- <key strategy change>

### Perception Prompts That Worked
| Object | Prompts | Notes |
|---|---|---|

### Failure Modes Eliminated
```
```

```
- <approach that failed and why>

### Generalizable Patterns
- <anything worth adding to the skill library>

### Skill Library Updates Made
- <if any skills were updated during this run>
```

### Return Format

```text
SUITE: <suite>
TASK: <task>
DEVICE: <N>
Baseline rate: <from actor pipeline>

Stage 1:
  Best candidate: <candidate_X> at iter_NN
  Best pass rate: <N>/<DEBUG_TOTAL> (<pct>
  Stopping reason: <solved|max_iterations|blocked>
  Iterations run: <N>
  Run dir: <RUN_DIR>

Stage 2: <N>/<HELD_OUT_TOTAL> (<pct>

Key findings:
  - <strategy that worked vs baseline failure mode>
  - <perception prompts or grasp parameters>
  - <generalizable pattern for the skill library>
```

## E.5. 初始技能库模板

grasp initial SKILL.md

```
---
name: grasp
description: Structural template for pick-and-place programs - topdown-quat construction, pre-grasp
     approach, lower-close sequence, lift-transport-place pattern. No task-specific parameters; those grow
      through experiment.
---

# Grasp - Structural Template for Pick-and-Place

> **Purpose:** Code skeletons that every pick-and-place task reuses. No task-specific z-offsets or yaws -
     those go in the per-task `task_code.py`.
> **Ownership:** Coordinator adds generalizable patterns. Subagents do not edit this file.

---
```

```
## Top-Down Quaternion

```python
import numpy as np
from scipy.spatial.transform import Rotation

def make_topdown_quat(yaw_deg=0):
    """Build a top-down end-effector quaternion (xyzw→wxyz convention used by solve_ik)."""
    R = Rotation.from_euler('z', yaw_deg, degrees=True).as_matrix() @ \
        np.array([[1, 0, 0], [0, -1, 0], [0, 0, -1]])
    q = Rotation.from_matrix(R).as_quat()  # scipy returns xyzw
    return np.array([q[3], q[0], q[1], q[2]])  # reorder to wxyz
```

**Yaw selection:** default 0. If the grasp fails because the gripper fingers collide with a neighboring
     object, try 90 or 45 degrees.

---

## Standard Pick-and-Place Skeleton

```python
# Assume: obj_center, obj_mask, tgt_center, tgt_pts already localized (see localize skill)
# Assume: depth, K (intrinsics), E (camera extrinsic pose_mat) from get_observation()

# 1. Plan grasp via GraspNet on object mask
# 2. Pre-grasp approach: gripper above object, open
# 3. Lower to grasp pose, close gripper
# 4. Lift (avoid dragging object sideways into neighbors)
# 5. Move above target (same z as lift)
# 6. Lower to release height (surface_z + margin)
# 7. Release
# 8. Let physics settle (important for drop-success reading)
    ... (skeleton code omitted)
```

---

## Why `select_top_down_grasp`?

`plan_grasp` returns grasps in camera frame, often angled or side-approach. We want top-down for table-top
      pick-and-place. The selector transforms to world frame and filters for top-down direction (z-axis
     roughly anti-aligned with gravity). When it returns None (no valid top-down), fall back to `E @
     grasp_poses[grasp_scores.argmax()]`.

---

## When to NOT Use This Template

- **Shelves / drawers / microwaves** - use `manipulation` skill, not this.
- **Stacking** - need stacking-specific surface_z logic (stack on top of existing object, not table).
```

```
- **Narrow containers (baskets with walls)** - need yaw that avoids wall collision; release height above
    basket rim.

---

## Per-Object Grasp Registry (grows per chunk)

Discovered per-object grasp parameters. Coordinator adds entries from `findings.md`.

| Object | z_offset | yaw | Notes | First Seen In |
|---|---|---|---|---|

<!-- PLACEHOLDER: Registry starts empty. Populated by coordinator after each chunk. -->
<!-- Format: object name | z_offset from OBB top | yaw_deg if non-zero | why | task that validated it -->

---

## Anti-Patterns

- `set_joint_qpos` to "teleport" the gripper into grasp pose - forbidden.
- Hardcoded grasp positions from task BDDL - forbidden (reading .bddl is banned).
- Lifting to fixed absolute z (e.g., z=0.9) - use `grasp_pos[2] + 0.15` to be scene-relative.
- Skipping the pre-grasp `z_approach=0.15` - causes gripper collisions with neighbors.
- Single-try `select_top_down_grasp` without `argmax` fallback - some scenes have no valid top-down grasp.

---

## Debugging Grasp Failures

| Symptom | Likely Cause | Check |
|---|---|---|
| Gripper closes on air | z too high (didn't descend to surface) | Print `grasp_pos[2]` vs. `surface_z`;
    use OBB `extent[2]/2` below top |
| Grasp succeeds but drops during lift | Gripper half-closed on slippery/elongated object | Try
    perpendicular yaw (90 degrees) or fingers-along-long-axis approach |
| `plan_grasp` returns empty | Mask too small or below grasp threshold | Dilate mask; log `mask.sum()` -
    should be >200 px |
| `solve_ik` returns None | Pose out of workspace | Lower z by 5cm, re-check x/y reachable from robot base
     |
```

localize initial SKILL.md

```
---
name: localize
description: Object localization via SAM3 - prompting strategies, multi-prompt fallback, 3D centroid
    extraction, disambiguation, per-object prompt registry. Grows through experiment.
---

# Localize - SAM3 Prompting & Object Localization

> **Purpose:** This skill tracks working SAM3 prompts per object + the standard localization helpers.
```

```
> **Ownership:** Coordinator adds entries to the prompt registry after reviewing subagent findings.
     Subagents do NOT edit this file.
---

## Standard Localization Helper

```python
import numpy as np

def localize_object(rgb, depth, K, E, prompts):
    """Try prompts in order, return (center, pts, mask) for first hit with >=10 points."""
    depth_img = depth[:, :, 0] if len(depth.shape) == 3 else depth
    if isinstance(prompts, str):
        prompts = [prompts]
    for prompt in prompts:
        masks = segment_sam3_text_prompt(rgb, prompt)
        if not masks:
            continue
        best = max(masks, key=lambda d: d["score"])
        mask = best["mask"].astype(np.uint8)
        pts = mask_to_world_points(mask, depth_img, K, E)
        if pts is None or len(pts) < 10:
            continue
        center = get_oriented_bounding_box_from_3d_points(pts)["center"]
        return center, pts, mask
    return None, None, None
```

**Usage:**
```python
obs = get_observation()
cam = obs["agentview"]
center, pts, mask = localize_object(
    cam["images"]["rgb"], cam["images"]["depth"],
    cam["intrinsics"], cam["pose_mat"],
    ["<specific prompt>", "<fallback prompt>"]
)
if center is None: raise RuntimeError("Object not found")
```

**Why OBB center over mean:** `get_oriented_bounding_box_from_3d_points` gives a more robust center for
     elongated/occluded objects than `pts.mean(axis=0)`.

---

## Fallback: Molmo Pixel Grounding

When SAM3 returns no masks even for sensible prompts, try Molmo to get a pixel, then convert with SAM3
     point prompt:

```python
def localize_via_molmo(rgb, depth, K, E, phrase):
```
```

```
    px = point_prompt_molmo(rgb, phrase)  # (x, y) or None
    if px is None: return None, None, None
    masks = segment_sam3_point_prompt(rgb, [px])
    if not masks: return None, None, None
    mask = masks[0]["mask"].astype(np.uint8)
    depth_img = depth[:, :, 0] if len(depth.shape) == 3 else depth
    pts = mask_to_world_points(mask, depth_img, K, E)
    if pts is None or len(pts) < 10: return None, None, None
    return get_oriented_bounding_box_from_3d_points(pts)["center"], pts, mask
```

---

## Disambiguation: Two Similar Objects in Scene

When a scene contains two visually similar objects, `max(score)` often picks the wrong one. Use **bbox pixel area** or **bbox y-position** (upper = farther):

```python
def select_by_smallest_bbox(masks, max_cy=None, min_area=50):
    """Pick mask with smallest bbox pixel area (most compact matching shape).
    max_cy: if set, only accept masks with bbox center-y below this row."""
    ... (implementation omitted)
```

**Key:** `area_pct` in SAM3 result dicts clamps to 99.00 for all masks - compute area from `box` directly.

---

## Disambiguation: By 3D Position (when two objects share prompts)

```python
# Example: "black bowl" returns both bowls, want the one at the front
    ... (implementation omitted)
```

---

## Prompt Registry (grows per chunk)

Discovered working prompts, indexed by object. Coordinator appends entries after each chunk review.

**Format per row:** object | priority-ordered prompts | which task validated it | notes.

| Object | Working Prompts | First Seen In | Notes |
|---|---|---|---|

<!-- PLACEHOLDER: Registry starts empty. Populated by coordinator after each chunk. -->
<!-- Format: object name | prompts in priority order (quoted) | first task that validated | disambiguation notes if needed -->

---

```
## How to Add a New Prompt (Coordinator Only)

When a subagent's `findings.md` "Generalizable Patterns" section mentions a new working prompt:

1. **Verify generalizability** - would this prompt plausibly work for >=2 task types? If it's ultra-
     specific to one task, consider keeping it inline in `task_code.py` instead of here.
2. **Append to registry** - one table row, priority-ordered list of prompts.
3. **Add a disambiguation note** only if the task exposed a specific confusion (e.g., two bowls in frame).
4. **Keep examples short** - a full code block per prompt is overkill. The helper pattern stays at the top
      of this file; the registry is just prompt strings.

---

## Anti-Patterns (Do NOT Do These)

- `body_xpos` for object localization - forbidden. If you're tempted, the perception is wrong; fix the
     prompt.
- Hardcoded pixel coordinates - always derive from SAM3 or Molmo.
- Caching object positions across seeds - re-localize every episode; objects are randomized.
- Using `segment_sam3_text_prompt` mask dict without `.astype(np.uint8)` conversion - silent failures
     downstream.
```

manipulation initial SKILL.md

```
---
name: manipulation
description: Non-pick-and-place manipulation patterns - drawer opening/closing, knob/switch turning,
     pushing, articulated-object interactions. Grows with validated parameters per object type.
---

# Manipulation - Articulated Objects & Non-Prehensile Actions

> **Purpose:** Tracks validated parameters for articulated manipulation. Approaches are discovered through
      experiment and recorded in the registry below.
> **Ownership:** Coordinator adds validated parameters per object-type. Subagents do not edit.
---

## Examples of Task Types Requiring This Skill

- **Drawer open/close** - pull/push a drawer by its handle
- **Knob/switch turn** - rotate a stove knob or flip a switch to trigger a state change
- **Hinged door** - open a microwave, cabinet door, or similar hinged panel
- **Non-prehensile push** - slide an object to a target without grasping it

These require non-top-down approach angles, arc motions, or contact-without-grasp patterns.

---

## Parameter Registry (grows per chunk)
```

```
Validated parameters per object / action. Coordinator adds after verifying from `findings.md`.

| Object / Action | Parameters | First Seen In | Notes |
|---|---|---|---|

<!-- PLACEHOLDER: Registry starts empty. Populated by coordinator after each chunk. -->
<!-- Format: object/action | key parameters (distances, angles, offsets, approach direction) | first task
     | notes on what failed before -->

---

## Anti-Patterns

- `set_joint_qpos` to force a drawer/door open - forbidden. Must use physical manipulation.
- Reading BDDL to get pull distances or knob angles - forbidden. Discover via OBB or trial-and-error.
- Forgetting to retreat after manipulation - arm stays inside drawer/microwave, blocks next perception.
```

transport initial SKILL.md

```
---
name: transport
description: Motion patterns for moving the end-effector between poses - multi-step waypoints, safe
     transit, interpolated Cartesian moves, collision avoidance. Grows with discovered patterns.
---

# Transport - End-Effector Motion Between Poses

> **Purpose:** Transit patterns after a grasp is achieved and before a place/release. Keeps the object
     above clutter, avoids scene collisions, handles narrow workspaces.
> **Ownership:** Coordinator adds validated waypoint patterns. Subagents do not edit.
---

## Basic Lift-Transit-Descend

```python
# Precondition: object grasped (close_gripper done), grasp_pos known, quat known
# Target: tgt_center (3D world coordinate), surface_z (target surface height)

# Waypoint 1: lift straight up (avoid dragging)
lift_z = grasp_pos[2] + 0.15
joints = solve_ik([grasp_pos[0], grasp_pos[1], lift_z], quat.tolist())
if joints is not None: move_to_joints(joints)

# Waypoint 2: move laterally at lift height (avoid hitting other objects on surface)
joints = solve_ik([tgt_center[0], tgt_center[1], lift_z], quat.tolist())
if joints is not None: move_to_joints(joints)

# Waypoint 3: descend to release height
release_z = surface_z + 0.05
joints = solve_ik([tgt_center[0], tgt_center[1], release_z], quat.tolist())
if joints is not None: move_to_joints(joints)
```
```

```
```

**Why 3 waypoints, not direct goto_pose:** a direct move from grasp to release tends to take a diagonal path that clips tall objects, basket walls, or the drawer opening. Three cardinal waypoints (up → over → down) keeps the object in free air during transit.

---

## Home Reset Between Subtasks

For long-horizon tasks (LIBERO-Long-Pro), reset the arm between subtasks to a known-safe config:

```python
def safe_reset_between_subtasks():
    open_gripper()
    goto_home_joint_position()
    for _ in range(3): get_observation()  # let physics settle
```

**When to call:** after releasing subtask-1's object, before starting subtask-2's localization. Prevents subtask-2 from trying to perceive through the arm in frame.

---

## Far-Workspace Safety: `solve_ik` over `goto_pose`

`goto_pose` is unreliable in the far workspace (e.g., back of counter, far side of table). Use `solve_ik` + `move_to_joints` for these transits:

```python
# UNRELIABLE (far workspace):
# goto_pose(far_target_pos, quat)

# RELIABLE:
joints = solve_ik(far_target_pos.tolist(), quat.tolist())
if joints is None:
    # Try slightly closer z or x
    fallback = far_target_pos.copy(); fallback[2] -= 0.02
    joints = solve_ik(fallback.tolist(), quat.tolist())
if joints is not None:
    move_to_joints(joints)
else:
    raise RuntimeError(f"IK infeasible at {far_target_pos}")
```

---

## Waypoint Registry (grows per chunk)

Validated multi-step transit patterns for scenes where direct motion fails.

```
| Scenario | Pattern | First Seen In | Notes |
|---|---|---|---|

<!-- PLACEHOLDER: Registry starts empty. Populated by coordinator after each chunk. -->
<!-- Format: scenario description | waypoint pattern used | first task | why default 3-waypoint failed -->

---

## Anti-Patterns

- Single `goto_pose` across the whole scene - clips clutter. Use waypoints.
- Lift waypoint at same z as grasp_pos - drags the object sideways.
- Descending straight into a container without a rim-clearance check - grasp collides with walls.
- Forgetting `for _ in range(3): get_observation()` after `open_gripper()` - object doesn't settle; next
    observation is stale.
- Using `goto_pose` in far workspace where `solve_ik`+`move_to_joints` is required.
```